

```
@'
"2026-08-10","Ben Swartz","Claude Web","Claude Opus 5","Planning; technical guidance;
'| @ | Add-Content -Path evidence\ai\registers\ben-swartz.csv -Encoding utf8
```

Check it landed on its own line:

```
powershell
```

```
Get-Content evidence\ai\registers\ben-swartz.csv | Select-Object -Last 2
```

Then export today's session as `2026-08-10-continuous-project-session.pdf` into `evidence\ai\transcripts\ben-swartz\`.

You're on `chore/105-measure-database-latency`, so it can go in that branch rather than needing its own:

```
powershell
```

```
git add evidence/ai/registers/ben-swartz.csv evidence/ai/transcripts/ben-swartz/
git commit -m "docs(evidence): register AI usage for 10 August" -m "Refs #105
Assisted-by: Claude-Web[Claude Opus 5]"
git push
```

done, next?

Comment on #63, then #43. Both are quick and both leave a clear record for whoever picks them up.

#63:

```
markdown
```

Seed data and the invalid submission examples are delivered by the pull request linked above. Expected figures for the reference fixture are documented in ``evidence/validation/729307-published-figures.md`` and asserted by ``apps/backend/scripts/validate-match.ts``, which derives every figure by SQL over ``delivery_current`` and reads nothing from the source file.

The remaining criterion is that those assertions run as part of the automated suite rather than on demand. That is currently blocked: ``assertSafeTestDatabase`` correctly requires a separate database whose name contains "test", and no such database exists. Creating one on the current instance would add storage against a limit ADR-003 already records as unresolved.

Two further gaps are recorded in the validation document itself and remain open: the per-type extras breakdown was derived from the database rather than read from the published scorecard, because the site blocks automated access; and the running score checkpoints and powerplay figures are recorded but not asserted.

This should be revisited once the storage decision in ADR-003 consequence 4 is taken.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

Tick on #63: the dataset criterion, the ordered events criterion, the expected-statistics criterion, and repeatable seed loading. Leave the invalid submission example ticked too — that's delivered.

#43:

markdown

Partly implemented by #27. ``app_user`` keys on ``(auth_provider, auth_subject)`` with an ``application_role`` column, so an authentication identity maps to one application account independently of the provider.

Outstanding: approved-submitter status, submission scope linked to the competition model, and the constraints enforcing both.

The scope model is undecided and should be settled before this is implemented. Whether a scope is a competition, a season, or a set of fixtures determines the schema here and constrains #44, #45 and #51, none of which can be completed without it.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

Tick nothing on #43, or at most the public read access criterion — and even that is generous, since no endpoints exist to enforce it.

Post both, then that's everything outstanding except the two PRs waiting on review.

done, now we close 46 and i can tick everything?

Not yet — the formatting PR is still open. #46's own criteria include the work being reviewed through a Pull Request, and that PR is the last piece of #46's delivery.

Check whether it merged while you were working:

```
powershell  
  
git fetch origin  
git log --oneline origin/main -3
```

If the formatting commit is in `main`, then yes: tick everything on #46 and close it.

If it hasn't merged, tick the delivered criteria now — tables, stable identifiers, relationships, fixture representation, seed data, ERD — and leave the PR criterion unticked until it lands. Then close.

Paste the log.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin  
>> git log --oneline origin/main -3  
remote: Enumerating objects: 1, done.  
remote: Counting objects: 100% (1/1), done.  
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Unpacking objects: 100% (1/1), 324 bytes | 40.00 KiB/s, done.  
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool  
 489e885..1f9a3aa main    -> origin/main  
1f9a3aa (origin/main, origin/HEAD) Merge pull request 'feat/46-add-development-  
seed-data' (#101) from feat/46-add-development-seed-data into main  
8f55774 Merge remote-tracking branch 'origin/main' into feat/46-add-  
development-seed-data  
363832f docs(db): add an entity relationship diagram generated from the schema  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

PR #101 merged. But check whether it included your formatting commit — you pushed that after the earlier merge, and the log shows `8f55774` as the merge-from-main, with your Prettier commit (`9b96bac`) not visible in these three lines.

```
powershell  
  
git log --oneline origin/main -8
```

If `chore(format): apply Prettier and exclude source match data` is in there, #46 is fully delivered — tick everything and close.

If it isn't, `main` still fails `format:check` on the seed files and `.prettierignore` is missing, meaning the next person to run Prettier reformats the Cricsheet data. That would need chasing before you close.

Paste it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git log --oneline origin/main -8
1f9a3aa (origin/main, origin/HEAD) Merge pull request 'feat/46-add-development-
seed-data' (#101) from feat/46-add-development-seed-data into main
8f55774 Merge remote-tracking branch 'origin/main' into feat/46-add-development-
seed-data
363832f docs(db): add an entity relationship diagram generated from the schema
3944ed9 feat(db): add invalid submission examples and enforce registry resolution
5576278 feat(db): add repeatable development seed data
489e885 Merge pull request '#39 Add Google authentication interface' (#100) from
feat/39-authentication-ui into main
c3bf64d fix(frontend): integrate authentication with public browsing
01bf5d5 Merge origin/main into feat/39-authentication-ui
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Your formatting commit `9b96bac` is **not** in `main`. PR #101 merged the earlier three commits only — it was created before you pushed the Prettier fix, or merged from a snapshot that predates it.

So `main` right now:

- fails `format:check` on the seed files and both READMEs
- has no `.prettierignore` entry for the match data, meaning the next `prettier --write` reformats the Cricsheet files and silently changes every seeded submission's provenance hash

That second one is the real risk, and it'll happen the first time anyone runs the formatter to fix their own work.

Confirm the commit still exists locally:

```
powershell
git log --oneline --all | Select-String "apply Prettier and exclude"
```

Then get it onto a fresh branch off current `main` and open a PR:

```
powershell
```

```
git switch main
git pull origin main
git switch -c chore/46-prettier-baseline-for-seed-files
git cherry-pick 9b96bac
git push -u origin chore/46-prettier-baseline-for-seed-files
```

Paste the output — the cherry-pick may conflict if anything else touched `.prettiignore`.

Don't close #46 yet. It isn't fully delivered until this lands.

```
PS C:\Users\BenSwartz\S
port-Analytics-Tool> git lo
g --oneline --all | Select-St
ring "apply Prettier and ex
clude" 9b96bac chore(for
mat): apply Prettier and...
```

PASTED

Cherry-pick clean, pushed. Open the PR:

Title: `#46 Apply Prettier to seed files and exclude source match data`

markdown

Purpose

The seed files merged in #101 were written before the Prettier baseline landed in `main` and do not pass `format:check`. This applies Prettier to them and excludes the source match data from the formatter.

Changes

- Prettier applied to `seed.ts`, both seed READMEs and `erd.md`.
- `.prettierignore` extended to exclude `database/seeds/matches/` and the invalid example JSON files.

Testing

`structure:check`, `format:check` and `lint` pass.

Known Limitations

The Cricsheet match files are deliberately excluded from Prettier rather than reformatted. They are committed verbatim so that the SHA-256 recorded in `submission.source\_sha256` matches the bytes the source published; reformatting them would silently change the provenance hash of every seeded submission. Until this merges, running `prettier --write` on the repository will do exactly that.

Related Issue

Refs #46

Two things from that pull worth noting:

Someone amended `evidence/validation/729307-published-figures.md` — 82 lines changed. That's your validation document. Worth reading what changed, since the figures in it are what `validate-match.ts` asserts against.

main gained a lot today — the public-read API, repositories, contracts, e2e tests, an OpenAPI spec. Dean's question this morning about linking data to the frontend now has an answer: the endpoints exist.

Once this PR merges, #46 can close.

[14:56, 10/08/2026] Ben Swartz: Heads up for anyone writing endpoints — every DB query from here is ~185ms round trip. Measured it, median 181ms over 20 samples (script's in #105, npm run db:latency). That's distance to the database region, not slow queries — indexing won't help. An endpoint doing 3 sequential queries starts at 555ms before doing anything useful. Worth designing for fewer round trips per request. Same reason ingestion is 68s a match. Could someone else run it and compare? If you get 40ms it's my connection and I'm

wrong.

[14:58, 10/08/2026] Shayna Underslak: Heads up — npm run check is failing on main at the moment, two separate things, neither of them anyone's branch:

Backend typecheck fails with "Cannot find module '@sport-analytics/contracts'" unless you build contracts first. check runs typecheck before build so a fresh clone always fails here. Workaround for now:

npm run build --workspace=@sport-analytics/contracts

17 frontend tests fail in beforeEach with "window.localStorage.clear is not a function" — App.test.tsx and PublicBrowsePages.test.tsx. Something in the test setup is stubbing localStorage without a clear method.

Flagging because we've got no CI runners, so check is the only gate we have. If it's red by default we'll all start ignoring it and something real will slip through.

Who owns the fr

i have tutoring now but i can fix the public read after

will u just make it an issue or something and assign me and then ill do it in like an hourish

[15:01, 10/08/2026] Shayna Underslak: otherwise ill just give chat what claude said and tell it to make an issue, whatever's easier

Show more

Two things worth noticing there.

Shayna sent your message. The `npm run check` text she posted at 14:58 is the draft I wrote for you, verbatim — including the trailing "Who owns the fr" where it got cut. So either you forwarded it to her and she reposted, or it got pasted from somewhere. Not a problem, but she's now the apparent author of a finding you made, and she's offering to fix a bug she may not have diagnosed.

She's asking you to create an issue and assign her. That's fine and easy. Here it is:

Title: `Frontend tests fail on window.localStorage.clear`

Type: Bug.

markdown

Description

Seventeen frontend tests fail in `beforeEach` with
`TypeError: window.localStorage.clear is not a function`.

Affected files:

- `apps/frontend/src/App.test.tsx` (9 tests)
- `apps/frontend/src/pages/PublicBrowsePages.test.tsx` (8 tests)

Both call `window.localStorage.clear()` in `beforeEach`. Something in the test environment provides a `localStorage` object without a `clear` method.

Reproduction

`npm run test --workspace=@sport-analytics/frontend`

Observed on `main` at commit 1f9a3aa on 10 August 2026.

Motivation

`npm run check` is the only merge gate the project has, because the repository has no CI runners. While it fails on `main` for reasons unrelated to any given branch, members cannot use it to judge their own work, and a real failure will be mistaken for this one.

Acceptance Criteria

- [] All seventeen tests pass
- [] `npm run test --workspace=@sport-analytics/frontend` passes on a clean checkout
- [] The cause is identified rather than worked around by removing the `clear()` calls

Evidence

Test output before and after, and the linked Pull Request.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Claude Opus 5].

Labels: `type: bug`, `area: frontend`, `priority: high`. Milestone Sprint 1. Assign Shayna.

The contracts build-ordering problem is separate and smaller — worth its own issue, but it can wait.

i made it, write the whatsapp to her saying i made the issue for her

A Straightforward B Shorter

Made the issue and assigned you — it's got the two failing test files, the repro command, and the commit I saw it on. Labelled bug, Sprint 1, high priority.

Only thing I'd say is don't just delete the `clear()` calls to make it green — something's stubbing `localStorage` without a `clear` method and it's worth knowing what.

The contracts build-order thing is separate and smaller, I'll do that one.

Enjoy tutoring.



Both offer to take the contracts build-ordering issue — say if you'd rather not commit to that and I'll cut the line.

That one is genuinely small: `npm run check` runs `typecheck` before `build`, so a fresh clone fails until contracts is built. The fix is probably reordering the script or adding a contracts build step ahead of typecheck. Fifteen minutes, and it removes half the noise from the only gate the team has.

[#46](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/issues/46) Apply Prettier to seed files and exclude source match data #107

Open

[BenSwartz](https://sdp.ms.wits.ac.za/BenSwartz) wants to merge 1 commits from `chore/46-prettier-baseline-for-seed-files` into `main`

[Conversation0](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/107)[Commits1](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/107/commits)[Files Changed5](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/107/files)+23 -16

[BenSwartz](https://sdp.ms.wits.ac.za/BenSwartz) commented [14 minutes ago] (https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/107#issue-225)

Member

Purpose

The seed files merged in [#101](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/issues/101) were written before the Prettier baseline landed in `main` and do not pass `format:check`. This applies Prettier to them and excludes the source match data from the formatter.

Changes

- Prettier applied to `seed.ts`, both seed READMEs and `erd.md`.
- `.prettierignore` extended to exclude `database/seeds/matches/` and the invalid example JSON files.

Testing

`structure:check`, `format:check` and `lint` pass.

Known Limitations

The Cricsheet match files are deliberately excluded from Prettier rather than reformatted. They are committed verbatim so that the SHA-256 recorded in `submission.source_sha256` matches the bytes the source published; reformatting

them would silently change the provenance hash of every seeded submission. Until this merges, running `prettier --write` on the repository will do exactly that.

Related Issue

Refs [#46](https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/issues/46)

[BenSwartz](https://sdp.ms.wits.ac.za/BenSwartz) added 1 commit 14 minutes ago [chore(format): apply Prettier and exclude source match data]

(https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/commit/19cc9adaa36ef73b66eefb8e9a925391719862a2)[19cc9adaa3]

(https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/commit/19cc9adaa36ef73b66eefb8e9a925391719862a2)

Some checks are pending

Continuous Integration / quality (pull_request) Waiting to run

[Details](<https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/actions/runs/114/jobs/0>)

This pull request doesn't have enough required approvals yet. 0 of 1 official approvals granted.

This branch is out-of-date with the base branch

The changes on this branch are already on the target branch. This will be an empty commit.

View command line instructions

Checkout

From your project repository, check out a new branch and test the changes.

```
git fetch -u origin chore/46-prettier-baseline-for-seed-files:chore/46-prettier-baseline-for-seed-files
```

```
git checkout chore/46-prettier-baseline-for-seed-files
```

Merge

Merge the changes and update on Gitea.

Warning: This operation cannot merge pull request because "autodetect manual merge" is not enabled.

```
git checkout main
```

```
git merge --no-ff chore/46-prettier-baseline-for-seed-files
```

```
git push origin main
```

WritePreview

Drop files or click here to upload.

No Reviewers

Still in progress? Add WIP: prefix

No Label

No Milestone

No project

No Assignees

1 Participants

Notifications

Due Date

No due date set.

Dependencies

No dependencies set.

Reference: git-push-pray/Sport-Analytics-Tool#107

[Powered by Gitea](<https://about.gitea.com/>)Version: 1.24.7

Page:172msTemplate:22ms

Show more

"The changes on this branch are already on the target branch. This will be an empty commit."

So someone already applied the Prettier fix to `main` while you were working. Your cherry-pick is now a no-op.

Verify before doing anything:

```
powershell
```

```
git fetch origin
```

```
git show origin/main:.prettiignore
```

If `database/seeds/matches/` is in there, the fix landed and you should close #107 as unnecessary — with a comment saying why, not silently.

If it isn't, then Gitea is comparing against something stale and the banner is misleading. Paste the output.

Either way, also run:

```
powershell

git checkout origin/main -- . 2>$null; npm run format:check
```

— actually, simpler and non-destructive:

```
powershell

git switch main
git pull origin main
npx prettier --check .
```

That tells you directly whether `main` passes now. Paste both.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin
>> git show origin/main:.prettiignore
remote: Enumerating objects: 70, done.
remote: Counting objects: 100% (44/44), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 25 (delta 15), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (25/25), 4.96 KiB | 48.00 KiB/s, done.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
* [new branch] feat/102-auth-callback-handling -> origin/feat/102-auth-callback-handling
_1f9a3aa..77774ec main -> origin/main
node_modules
dist
coverage
site
*.min.js
package-lock.json
.venv/
# Preserve AI transcript evidence verbatim
evidence/ai/transcripts/
# Third-party source data. These files are committed verbatim so that the
# submission hash matches what the source published. They must not be
reformatted.
database/seeds/matches/
database/seeds/invalid/*.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Your `.prettiignore` entry is in `main` — someone merged the fix while you were working. #107 is genuinely redundant.

Close it with a comment rather than silently:

markdown

Already applied to `main`. Closing as redundant.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

Then delete the branch:

powershell

```
git switch main
git pull origin main
git branch -D chore/46-prettier-baseline-for-seed-files
git push origin --delete chore/46-prettier-baseline-for-seed-files
```

Then confirm `main` actually passes formatting now:

powershell

```
npx prettier --check .
```

If that's clean, **#46 is fully delivered** — tick all criteria and close it.

Paste the prettier output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npx prettier --check .
Checking formatting...
[warn] apps/backend/scripts/latency.ts
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

One file, and it's yours — `latency.ts`, on your `chore/105` branch and not yet formatted.

Fix it there:

powershell

```
git switch chore/105-measure-database-latency
git pull origin main
npx prettier --write apps/backend/scripts/latency.ts
npx prettier --check .
```

If clean:

powershell

```
git add apps/backend/scripts/latency.ts
git commit -m "chore(format): apply Prettier to the latency script" -m "Refs #105"
git push
```

Paste the output. Note `git pull origin main` while on your branch will merge main in — expected, since main has moved several times today.

Then open the PR for #105 if you haven't, and #46 can close.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git add
apps/backend/scripts/latency.ts
>> git commit -m "chore(format): apply Prettier to the latency script" -m "Refs
#105"
>> git push
[chore/105-measure-database-latency cdc9c16] chore(format): apply Prettier to
the latency script
1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 530 bytes | 265.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'chore/105-measure-database-latency':
remote: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/new/chore/105-measure-database-latency
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
* [new branch] chore/105-measure-database-latency -> chore/105-measure-
database-latency
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Pushed. Two things left, both quick.

1. Open the PR for #105 at the URL Gitea printed. Title and description from earlier — the one ending with the caveat that it's one machine, one location, and that another member's figure would confirm or refute it. Note the branch also carries your 10 August register row and transcript, so mention those in Changes:

markdown

Changes

- `apps/backend/scripts/latency.ts` and the `db:latency` script. It times connection setup once, then twenty sequential `SELECT 1` statements over the established connection, reporting min, median, mean and max.
- A 10 August row in `evidence/ai/registers/ben-swartz.csv` and the session transcript it cites.

2. Close #46. All criteria are now delivered: tables, stable identifiers, relationships, seed data, ERD, and both PRs merged. Tick everything and close, then move the board item to **Done**.

After that you're clear. Open: #63 and #43 both commented and correctly parked, #105 and #84 awaiting review, and the super-over issue in the backlog for Sprint 2.

someone had already merged 105...

Check what actually landed — the branch existed only moments ago, and your formatting commit went up after:

```
powershell

git fetch origin
git log --oneline origin/main -5
git log --oneline origin/main -3 -- apps/backend/scripts/latency.ts
```

Paste that. What I want to know is whether the merge included **cdc9c16** (the Prettier fix) or only the earlier commits — same trap as #46, where a PR merged a snapshot and left your last commit behind.

If the formatting commit is in, #105's work is done and you can tick and close it. If not, **main** fails **format:check** again on **latency.ts** and you'll need one more small PR.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin
>> git log --oneline origin/main -5
>> git log --oneline origin/main -3 -- apps/backend/scripts/latency.ts
77774ec (origin/main, origin/HEAD, main) Merge pull request '#105 Add a database
round-trip latency measurement' (#106) from chore/105-measure-database-
latency into main
891f95f Merge branch 'main' into chore/105-measure-database-latency
b5ce31d Merge pull request '#46 Apply Prettier to seed files and exclude source
match data' (#103) from feat/46-add-development-seed-data into main
48bef5f Merge branch 'main' into feat/46-add-development-seed-data
277379d docs(evidence): register AI usage for 10 August
6082a78 chore(db): add a database round-trip latency measurement
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

PR #106 merged, but it took `6082a78` — the original latency commit — not `cdc9c16`, your Prettier fix. Same pattern as #46: merged before your last push.

So `main` still fails `format:check` on `latency.ts`. Confirm:

```

powershell

git switch main
git pull origin main
npx prettier --check .

```

If it flags `latency.ts`, push the fix straight from a fresh branch:

```

powershell

git switch -c chore/105-format-latency-script
git cherry-pick cdc9c16
git push -u origin chore/105-format-latency-script

```

Then a one-line PR: `#105 Apply Prettier to the latency script`.

Worth naming the pattern when you next talk to the group: this is the third time today a PR has merged a snapshot that excluded your most recent commit. Reviewers are approving from the Files Changed view without refetching, so anything pushed after they open the page gets left behind. Cheap fix — refresh before merging — but it's cost you three follow-up PRs.

Paste the prettier output.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main
>> git pull origin main
>> npx prettier --check .
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
* branch      main      -> FETCH_HEAD

```

```
Already up to date.
Checking formatting...
[warn] apps/backend/scripts/latency.ts
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Confirmed. Push the fix:

```
powershell

git switch -c chore/105-format-latency-script
git cherry-pick cdc9c16
git push -u origin chore/105-format-latency-script
```

Then open the PR:

Title: #105 Apply Prettier to the latency script

markdown

Purpose

`apps/backend/scripts/latency.ts` was merged in #106 before the formatting commit on the same branch was pushed, so `main` currently fails `format:check` on it.

Changes

One line reformatted by Prettier.

Testing

`npx prettier --check .` passes on the whole repository with this applied.

Related Issue

Refs #105

Paste the push output. Once this merges, both #46 and #105 are genuinely complete and `main` passes formatting again.

both merged, i can close both?

Verify first — you've now been caught three times by merges landing short:

powershell

```
git switch main
git pull origin main
npx prettier --check .
```

If that comes back clean, yes. Close both, tick the criteria, move both board items to **Done**.

For #105 specifically, note it isn't fully "done" in the sense its criteria describe — the measurement is recorded and the script exists, but the acceptance criteria include recording the figure in ADR-003, stating a target response time, and the team deciding whether latency justifies revisiting the host. None of those has happened.

So either close #105 as "measurement delivered" and open a separate issue for the decision, or leave it open with the measurement criteria ticked and the decision ones outstanding. I'd leave it open — the decision is the point of the issue, and closing it because the script exists loses the thread.

#46 closes cleanly.

Paste the prettier output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main
>> git pull origin main
>> npx prettier --check .
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 340 bytes | 56.00 KiB/s, done.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
* branch main -> FETCH_HEAD
Show more
77774ec..1b6d44a main -> origin/main
Updating 77774ec..1b6d44a
Fast-forward
 apps/backend/scripts/latency.ts | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
Checking formatting...
All matched files use Prettier code style!
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Clean. **main** passes formatting.

Close #46. Tick everything, move the board item to **Done**.

Leave #105 open. Tick the measurement, leave the four decision criteria — recording in ADR-003, endpoint design, stated response target, and the host question. That's the substance of the issue and none of it has happened.

Delete your merged branches locally:

```
powershell
```

```
git branch -D chore/105-measure-database-latency chore/105-format-latency-script feat/
```

Where things stand: #46 closed, #84 and #105 open with work outstanding, #63 and #43 parked on decisions, the super-over issue in the backlog, and the localStorage bug assigned to Shayna.

Still unowned and now blocking several things: the storage decision, the submission scope model, and password reset and account deletion. All three are Tuesday.

[17:25, 10/08/2026] Shayna Unterslak: pls can u all make sure u are formatting your files b4 u push
theres a prettier functions thing
i fixed all the wrong ones this morning and i see there already more
[17:25, 10/08/2026] Dean Feldman: give ur chats the new zip folder
[17:25, 10/08/2026] Shayna Unterslak: do npm run check b4 pushing and if u see format errors ask AI how to fix them
[17:26, 10/08/2026] Shayna Unterslak: Heads up — npm run check is failing on main at the moment, two separate things, neither of them anyone's branch:
Backend typecheck fails with "Cannot find module '@sport-analytics/contracts'" unless you build contracts first. check runs typecheck before build so a fresh clone always fails here. Workaround for now:
Show more
npm run build --workspace=@sport-analytics/contracts
17 frontend tests fail in beforeEach with "window.localStorage.clear is not a function" — App.test.tsx and PublicBrowsePages.test.tsx. Something in the test setup is stubbing localStorage without a clear method.
Flagging because we've got no CI runners, so check is the only gate we have. If it's red by default we'll all start ignoring it and something real will slip through.
Who owns the fr
can someone whose getting errors when they run this pls send me the output of typing node -v in their terminal, oim getting no error
[17:27, 10/08/2026] Dean Feldman: can someone whose getting errors when they run this pls send me the output of typing node -v in their terminal, oim getting no error
i also dont get any errors
[17:28, 10/08/2026] Shayna Unterslak: chat says its may be bc of an error with node 25, but says i shouldnt make the fix until i know forsure if thats the reason
[17:33, 10/08/2026] Shayna Unterslak: @Dean Feldman whats your version?
[17:33, 10/08/2026] Dean Feldman: the latest
[17:34, 10/08/2026] Dean Feldman: higher than 22
[17:34, 10/08/2026] Shayna Unterslak: what does it sday when u node -v in terminal?
[17:34, 10/08/2026] Dean Feldman: v24.18.0
[17:47, 10/08/2026] Dean Feldman: guys we need the data to display for tommorow

Shayna's hypothesis is probably right, and you have the evidence to confirm it.

You're on Node 25.9.0. Dean is on 24.18.0. He gets no error; you do. That's exactly the pattern for a Node-version-dependent failure — and jsdom's `localStorage` implementation is a plausible casualty of a Node major bump. I flagged your Node version as a divergence risk this morning; this is it surfacing.

So the answer to Shayna's question is: yes, run `node -v` and tell her 25.9.0, alongside Dean's 24.18.0 and the fact that you see the failures and he doesn't. Two data points make it a hypothesis worth testing rather than a guess.

The fix isn't "downgrade Ben" — it's that the project should pin a Node version. Your stack is documented as Node 20, and you have three members on three different majors. An `.nvmrc` or an `engines` field in `package.json` makes this stop happening.

Dean's last line is the more urgent one. "We need the data to display for tomorrow" — with the public-read API now merged, that's plausible, but the shared database has four fixtures and 955 deliveries in it, seeded from `db:seed`. If he's expecting the full corpus, that's blocked on the storage decision and nobody has taken it.

Want a message covering both? The Node finding is quick and useful; the data expectation needs managing before tomorrow rather than at 9am.

write a private message to dean on whatsapp, chilled, explaining where we are

A Full picture B Shorter

Hey man, on the data for tomorrow — wanted to give you the actual picture so you're not surprised in the morning.

There is data in the DB, but it's 4 matches, 955 deliveries. That's the dev seed I set up today (npm run db:seed loads it, and it's repeatable so you can run it whenever). Enough to demo the endpoints properly, not enough to look like a real analytics platform.

The full corpus is ~14k matches / 3.2m deliveries, roughly 1.3GB in Postgres. Two things stop us loading it:

1. Supabase free plan is 500MB. ADR-003 flagged this back on the 5th and says we have to decide between paying, cutting scope, or moving provider before we bulk load. Nobody's owned that decision yet.
2. Ingestion is ~68 seconds a match right now, because every delivery is a separate round trip and the DB is in Frankfurt (185ms each way). Even 300 matches is 5+ hours.

So for tomorrow I'd plan on demoing with the 4 seeded fixtures. They're actually decent for it — one's got a super over, two have miscounted overs, and one's validated against a published scorecard. Shows the model handles the weird cases.

If you want more than that you need the storage cell made, which is a separate thing. Worth emailing tomorrow?



The first is worth the extra length — it gives him what he needs to plan tomorrow rather than just telling him no.

One thing I'd add if there's a stakeholder meeting tomorrow: the four seeded fixtures are genuinely a better demo than a partial corpus would be. They were chosen because they

break naive models — the super over, the two miscounted-over variants, and a fixture validated ball-by-ball against a published scorecard. "Here's why our event model is right" is a stronger story than "here's a lot of rows."



i think i fixed it, when u get a chance, will u merge my pr and test it on your comp
this is from shayna

Two things worth separating there.

Testing it is genuinely useful — you're the one who sees the failures, so you're the only person who can confirm the fix. Pull her branch and run the frontend tests before merging.

Merging it yourself is a problem. Your git methodology says a developer may not merge their own PR without approval from another member — which is satisfied here, you'd be the other member. But the same section requires the reviewer to confirm the implementation matches its stated purpose, and there's a specific thing to check: whether she fixed the cause or just removed the `clear()` calls. The issue's acceptance criteria say explicitly not the latter.

So: review it properly, then merge if it's right.

powershell

```
git fetch origin  
git branch -r | Select-String "localStorage|clear|test"
```

Paste that so we can find her branch, then check it out and run the tests.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin  
>> git branch -r | Select-String "localStorage|clear|test"  
remote: Failed to authenticate user  
fatal: Authentication failed for 'https://sdp.ms.wits.ac.za/git-push-pray/Sport-  
Analytics-Tool.git/'  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

The usual intermittent auth failure — retry:

powershell

```
git fetch origin  
git branch -r
```

Paste the full remote branch list; her branch may not have "localStorage" or "test" in the name, so better to see all of them.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin
>> git branch -r
remote: Enumerating objects: 87, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (55/55), done.
remote: Total 55 (delta 28), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (55/55), 991.13 KiB | 514.00 KiB/s, done.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
  1b6d44a..41ee2a2 main    -> origin/main
* [new branch]    fix/108-frontend-localstorage-tests -> origin/fix/108-frontend-
localStorage-tests
Show more
origin/HEAD -> origin/main
origin/chore/105-format-latency-script
origin/chore/105-measure-database-latency
origin/chore/98-format-issue-markdown
origin/ci/10-restore-green-ci
origin/docs/47-public-read-openapi-sync
origin/docs/59-openapi-baseline
origin/docs/73-align-architecture-roadmap
origin/docs/84-add-per-member-ai-register
origin/docs/84-register-9-august
origin/docs/cloudflare-pages-deployment-v2
origin/docs/dean-ai-evidence
origin/docs/final-cloudflare-documentation-updates
origin/feat/102-auth-callback-handling
origin/feat/46-add-development-seed-data
origin/feat/47-public-read-api
origin/feat/62-database-repositories
origin/feat/64-frontend-auth-state
origin/feat/89-public-landing-shell
origin/fix/108-frontend-localstorage-tests
origin/infra/15-azure-backend-deployment
origin/main
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

`fix/108-frontend-localstorage-tests`. Check it out and see what she actually changed:

powershell

```
git switch fix/108-frontend-localstorage-tests
git diff origin/main --stat
```

Paste that first. If the only changed files are the two test files, that's a warning sign — it would mean the `clear()` calls were removed rather than the cause fixed. If a setup file or config changed, she's found the real cause.

Then run the tests:

```
powershell
```

```
npm run test --workspace=@sport-analytics/frontend
```

Paste both.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch fix/108-frontend-
localstorage-tests
>> git diff origin/main --stat
branch 'fix/108-frontend-localstorage-tests' set up to track 'origin/fix/108-
frontend-localstorage-tests'.
Switched to a new branch 'fix/108-frontend-localstorage-tests'
 README.md | 2 +-
 apps/frontend/vite.config.ts | 10 +
 docs/requirements/sport-domain-definition.md | 92 +-
 evidence/ai/registers/gabriel-raz.csv | 5 -
 evidence/ai/registers/liora-rosenberg.csv | 3 -
 evidence/ai/registers/shayna-unterslak.csv | 3 +-
 ...2026-08-10-issue-102-oauth-callback-handling.md | 8766 -----
 ...on-foundation.md => AI transcript Issue #14.md} | 17 +-
 evidence/ai/transcripts/gabriel-raz/README.md | 2 +-
 ...08-10-issue-108-frontend-localstorage-tests.pdf | Bin 0 -> 1339975 bytes
10 files changed, 68 insertions(+), 8832 deletions(-)
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

The actual fix is one file — `apps/frontend/vite.config.ts`, +10 lines. That's the right place: a test-environment fix rather than deleting the `clear()` calls. Good sign.

But the diff carries a lot of unrelated damage:

- **Gabriel's register loses 5 rows, Liora's loses 3.** Deletions from other members' AI usage records.
- **An 8,766-line transcript is deleted** — Gabriel's issue #102 OAuth callback transcript.
- **A transcript is renamed** from a descriptive filename to `AI transcript Issue #14.md`.
- `docs/requirements/sport-domain-definition.md` changes by 92 lines.

Almost certainly her branch is behind `main` and the diff is showing reversals of things that landed after she branched — not deliberate deletions. But that's a guess, and if she merges without updating her branch, those deletions become real.

See the fix itself first:

```
powershell
```

```
git diff origin/main -- apps/frontend/vite.config.ts
```

Then run the tests:

```
powershell
```

```
npm run test --workspace=@sport-analytics/frontend
```

```
main
```

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git diff origin/main --
apps/frontend/vite.config.ts
diff --git a/apps/frontend/vite.config.ts b/apps/frontend/vite.config.ts
index feea22b..16520d8 100644
--- a/apps/frontend/vite.config.ts
+++ b/apps/frontend/vite.config.ts
@@ -1,6 +1,8 @@
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vitest/config';

+const nodeMajorVersion = Number(process.versions.node.split('.')[0]);
$show more
export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
@@ -15,5 +17,13 @@ export default defineConfig({
  environment: 'jsdom',
  setupFiles: ['./src/test/setup.ts'],
  globals: true,
+ poolOptions: {
+   forks: {
+     // Node 25+ exposes Web Storage globals that conflict with Vitest's
+     // jsdom environment. Disable Node's implementation in test workers
+     // so jsdom provides the browser Storage API.
+     execArgv: nodeMajorVersion >= 25 ? ['--no-experimental-webstorage'] : [],
+   },
+ },
+ },
```

24.

Press **q**

```
powershell  
  
npm run test --workspace=@sport-analytics/frontend
```

```
npm run test --workspace=@sport-analytics/frontend
```

Node 25.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run test --workspace=@sport-  
analytics/frontend
```

```
> @sport-analytics/frontend@0.1.0 test
> vitest run
RUN v2.1.9 C:/Users/BenSwartz/Sport-Analytics-Tool/apps/frontend
✓ src/App.test.tsx (13) 694ms
✓ src/api/client.test.ts (4)
✓ src/api/public-read.test.ts (3)
✓ src/pages/PublicBrowsePages.test.tsx (8) 822ms
✓ src/features/auth/AuthProvider.test.tsx (4)
Test Files 5 passed (5)
  Tests 32 passed (32)
Start at 17:59:20
Duration 5.78s (transform 843ms, setup 3.31s, collect 2.59s, tests 1.78s,
environment 13.23s, prepare 1.48s)
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Show more

32 passed, 0 failed, on the machine that was failing. The fix works.

But **don't merge yet**. Her branch is behind `main`, and merging as-is would revert other people's work — Gabriel loses 5 register rows and an 8,766-line transcript, Liora loses 3 rows, and `sport-domain-definition.md` goes backwards.

Approve, note the confirmation, and ask her to update the branch first. Comment on the PR:

markdown

Confirmed on Node 25.9.0, where the failures reproduce: all 32 frontend tests pass with this applied. The diagnosis looks right — Node 25 provides Web Storage globals natively, they take precedence over jsdom's, and Node's implementation lacks `clear`. Scoping the flag to Node 25 and above means it is a no-op for members on 24.

One thing before this merges. The branch is behind `main` and the diff against it reverts work that landed afterwards: five rows from `gabriel-raz.csv`, three from `liora-rosenberg.csv`, an 8,766-line transcript under `gabriel-raz/`, and 92 lines of `docs/requirements/sport-domain-definition.md`. Merging `main` into the branch should clear all of it.

Approving on the fix itself, subject to that.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

And a message so she sees it tonight rather than tomorrow:

"Tested on my machine — all 32 pass, so the Node 25 thing was it, nice one. Don't merge yet though, your branch is behind main and the diff reverts a chunk of Gabriel's and Liora's stuff. Merge main in first and it should be clean. Approved otherwise."

Hey — quick one on the data for tomorrow so you're not caught out.
There is data, but it's the dev seed: 4 matches, 955 deliveries. npm run db:seed loads it, safe to run anytime. Fine for demoing endpoints, not a full dataset.
Can't load the full corpus yet for two reasons — the free Supabase plan is 500MB and the corpus is ~1.3GB (ADR-003 flagged this on the 5th, decision still not made by anyone), and ingestion is 68s a match so it'd take hours regardless.
The 4 fixtures are decent for a demo though — super over, miscounted overs, one validated against a real scorecard. Shows the model handles the awkward stuff.
from Claude
how do we fix?
thats a converstaion between me and dean

He's asking how to fix it, so give him options with costs. Three real ones:

Load a subset that fits. 500 MB free, roughly 414 bytes per delivery all-in, so about 1.2M deliveries — call it 1,000 matches with headroom. At 68s each that's 19 hours of ingestion, which is the blocker, not the storage. Batching the ingester first would cut it to about an hour. That's the "no decision needed, just work" option.

Pay. Supabase Pro is around \$25/month for 8 GB. Six people splitting it is trivial money, and it removes the constraint entirely. Somebody has to own a card and the group has to agree.

Move the database to Azure, which is already in your deployment plan. Fixes storage *and* the 185ms latency, since it'd be in South Africa North with the app. Biggest win, biggest disruption, and ADR-003 would need rewriting.

Cut scope — two or three competitions instead of eleven. Cheapest, and nobody wants it.

My read: batching the ingester is worth doing regardless, because 19 hours is unworkable whatever you decide about storage. And the Azure move is the answer that solves two problems at once, but it's a group decision with real work behind it.

Want that as a message to Dean?

yes

A Options with costs B Shorter

Few options, roughly cheapest to biggest:

****1. Just load what fits.**** 500MB gets us about 1.2m deliveries, so ~1000 matches with headroom. No decision needed, no money. Problem is ingestion at 68s a match = 19 hours. Which brings us to...

****2. Batch the ingester.**** Right now it does one round trip per delivery, ~290 per match, and each one costs 185ms because the DB is in Frankfurt. Batch them into one insert per innings and it's ~6 round trips a match instead. 19 hours becomes about an hour. This is worth doing whatever else we decide — the Sprint 2 batch pipeline needs it anyway. Maybe 2 hours of work.

****3. Pay for Supabase Pro.**** ~\$25/month, 8GB. Six of us splitting it is nothing. Removes the limit entirely. Needs someone to own a card and the group to agree.

****4. Move the DB to Azure.**** It's already in our deployment plan. Fixes storage AND the 185ms latency, since it'd be in South Africa North with the app. Biggest win but biggest disruption — ADR-003 would need rewriting and someone has to actually do the migration.

****5. Cut scope**** — 2-3 competitions instead of 11. Cheapest, nobody wants it.

Mistake: do 3 regardless. It's needed either way. Then 2 or 4 as a group call. 4 is the better answer long-term.



One caveat to flag when you send it: the ~1000 matches figure rests on the 414 bytes per delivery estimate in ADR-003, which has never been verified. Your measured figure covers heap only and can't confirm it. So treat 1000 as an order of magnitude, not a number to plan against — and if you do batch the ingester, loading a few hundred matches would finally give you the real measurement the ADR asks for.

That's the argument for doing the batching first, incidentally: it's the one action that unblocks both the loading and the measurement that informs the decision.

[20:11, 10/08/2026] Nadav Sundy: Is there anything else we need to do before tomorrow?

[20:12, 10/08/2026] Shayna Underslak: i dont think our db is connected, but i also cant rilly run it locally 🤖 maybe someone else can confirm

[20:13, 10/08/2026] Nadav Sundy: I can't either @Dean Feldman can you check please

[20:13, 10/08/2026] Shayna Underslak: otherwise like we not getting getting marked tom, its just a client meeting, so we can figure it out as we go and know for next time



[20:13, 10/08/2026] Nadav Sundy: Also @Gabriel Raz @Ben Swartz can someone share the db access with my personal email nsundy0@gmail.com so I can see the backend if it

[20:14, 10/08/2026] Shayna Underslak: should we message on the other group and ask him if we still doing tom bc of the compulsory thing... coz acc to our methodology we need to, so we can confirm or maybe ask for 145ish?

[20:14, 10/08/2026] Shayna Underslak: what do ppl think

[20:14, 10/08/2026] Nadav Sundy: Yeah let's ask

[20:14, 10/08/2026] Shayna Underslak: k, ill ask

[20:17, 10/08/2026] Dean Feldman: I can't either @Dean Feldman can you check please

I can't either

[20:17, 10/08/2026] Shayna Unterslak: can we not link to azure at all?
[20:17, 10/08/2026] Dean Feldman: Spoke to @Ben Swartz, he said he'd look at it
[20:18, 10/08/2026] Nadav Sundy: Can't run it or its not connected
[20:18, 10/08/2026] Shayna Unterslak: surely we can link even tho it just wont be continuously updated
[20:18, 10/08/2026] Dean Feldman: can we not link to azure at all?
It should be connected, just watching spider man, can look when home 🤪
this ius the grouo, i just came back from a night out

Show more

Three of them can't connect. You can — you've been running against it all day. That's the useful thing you know that they don't.

The likely cause is boring: ADR-003 says every developer needs the pooler connection string and the CA certificate, both documented in `docs/development/setup.md` because neither is discoverable from the Supabase dashboard. Nadav was asking for the connection string this morning and got a placeholder pasted in the chat. If nobody's actually followed the setup doc, "not connected" means "no `.env`", not a broken database.

Nadav's request for database access on his personal email is a separate thing and I'd not action it tonight — dashboard access to the project is different from a connection string, and it's the sort of thing to do deliberately rather than at 8pm after a night out.

Quick reply that unblocks them without you doing work now:

"DB is definitely up, I've been running against it all day — seeded 4 fixtures into it this afternoon. If you can't connect it's almost certainly your .env. You need the pooler string and the CA cert, both are in docs/development/setup.md, neither is findable in the Supabase dashboard which is why they're documented. Try that and tell me the error if it still fails. Nadav — dashboard access is a different thing to a connection string, let's sort that properly tomorrow rather than tonight."

Worth adding: if they follow the doc and still fail, that's a real finding about the doc, and better to know before the client meeting than during it.

but its not connecting to the frontend website?
why?

I don't know, and I'd be guessing. But there are a few plausible causes and one is much more likely than the others.

The most likely: the deployed frontend has nowhere to call. The frontend talks to the backend over HTTP. If the backend isn't deployed to Azure, or is deployed but the frontend's API base URL points at localhost, then the site loads and every data request fails — which looks exactly like "the database isn't connected" even though the database is fine.

Second: the deployed backend has no `DATABASE_URL`. Environment variables set locally in `.env` don't exist in Azure App Service unless someone configured them there. Same for the CA certificate.

Third: it works and nobody's looked at the right URL. Cloudflare Pages hosts the docs site; the app is a different deployment.

The thing that would tell you in thirty seconds: open the deployed site, open browser devtools, look at the Network tab, and see what the failing request is and where it's going. A 404 on localhost is cause one. A 500 from a real backend URL is cause two.

That's a five-minute check tomorrow morning, not a tonight job — and it's Dean's territory since he owns the Azure deployment.

If you want, I can write the question for the group so someone with the deployment in front of them can answer it properly.

how do i check/fix it right now. it will be off the books

Diagnosing it now is fine — that's just looking. But "off the books" is where I'd push back, and not on principle.

Your Sprint 1 rubric is roughly 80% process. An unlogged fix pushed at midnight after a night out is the exact thing that shows up in a marker's eyes as inconsistent methodology — and it undoes some of the value of a day where you did everything properly. It also means the group doesn't learn what was wrong, and Dean, who owns the deployment, finds out by reading a diff.

The diagnosis costs you nothing and can be shared in the morning. Here's how:

Step 1 — find the deployed frontend URL. Do you have it? If not:

```
powershell
```

```
Select-String -Path docs\deployment\*.md -Pattern "azurewebsites|pages.dev|https://" |
```

Step 2 — open it, open devtools (F12), Network tab, reload. Look for failed requests. What you want is the URL it's calling and the status code.

Paste what you find and I'll tell you what it means. If it turns out to be a one-line environment variable in Azure, then at least you'll know, and it's a two-minute fix in the morning with a commit message attached.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path  
docs\deployment\*.md -Pattern "azurewebsites|pages.dev|https://" | Select-  
Object -First 20
```

docs\deployment\cloudflare_pages.md:63:<https://sports-analytics-tool.pages.dev>
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Only the docs site is there. No Azure URL in the deployment documentation at all — which is itself a finding: `docs/deployment/azure.md` and `azure-backend.md` exist, but neither records a deployed URL.

That points at cause one from my list. If the backend has never actually been deployed, the frontend has nothing to call, and no amount of environment fiddling fixes it tonight.

Check whether anything was deployed:

```
powershell
```

```
Select-String -Path docs\deployment\azure*.md -Pattern "http|url|endpoint|deployed|app"
```

And what CI does:

```
powershell
```

```
Select-String -Path .\gitea\workflows\*.yaml -Pattern "deploy|azure|webapp" -Context 2,2
```

Paste both.

My honest expectation is that you'll find a deployment plan and no deployment — in which case there's nothing to fix tonight, and the right move is a message to Dean before the meeting rather than a midnight commit.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
docs\deployment\azure*.md -Pattern "http|url|endpoint|deployed|app service" -
Context 1,1 | Select-Object -First 30
docs\deployment\azure-backend.md:4:
> docs\deployment\azure-backend.md:5:Azure App Service (Linux)
docs\deployment\azure-backend.md:6:
docs\deployment\azure-backend.md:12:
> docs\deployment\azure-backend.md:13:App Service
docs\deployment\azure-backend.md:14:
docs\deployment\azure-backend.md:26:| API_VERSION      | v1      |
> docs\deployment\azure-backend.md:27:| CORS_ALLOWED_ORIGINS | Frontend
Azure URL |
docs\deployment\azure-backend.md:28:| LOG_LEVEL      | info    |
docs\deployment\azure-backend.md:29:
> docs\deployment\azure-backend.md:30:Database credentials are configured
through Azure App
Service and are never committed.
docs\deployment\azure-backend.md:31:
docs\deployment\azure-fronted.md:4:
> docs\deployment\azure-fronted.md:5:Azure App Service (Linux)
```

```
docs\deployment\azure-fronted.md:6:
docs\deployment\azure-fronted.md:23:| ----- | ----- |
> docs\deployment\azure-fronted.md:24:| VITE_API_BASE_URL | Backend Azure
URL |
docs\deployment\azure-fronted.md:25:| VITE_APP_NAME   | Stat'sTheGame   |
docs\deployment\azure-fronted.md:31:
> docs\deployment\azure-fronted.md:32:The frontend is deployed through Gitea
Actions after
successful builds.
docs\deployment\azure.md:4:
> docs\deployment\azure.md:5:Before Milestone 1, prototype independent
deployment of the
frontend, backend health endpoint, database connection, and MkDocs site. Record
public URLs,
exact commands, secret handling, CORS/HTTPS behaviour, logs, health monitoring,
rollback
steps, and cost or student-credit impact.
docs\deployment\azure.md:6:
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

That confirms it. Every URL in those docs is a placeholder — "Frontend Azure URL", "Backend Azure URL". No actual deployed address anywhere. And `azure.md` frames the whole thing as work to do *before* Milestone 1, not work that's done.

Also: `feat/89-public-landing-shell`, `feat/47-public-read-api` and the auth work all merged into `main` today, but the deployment is a plan, not a thing.

So there is nothing to fix tonight. The frontend isn't failing to reach the database — the frontend isn't deployed, or if it is, the backend it would call isn't. Neither has a URL recorded. That's not a config error you can patch from your machine at midnight; it's a deployment that hasn't happened.

Which means the honest position for tomorrow's client meeting is: the API and frontend work locally, the deployment is outstanding. That's a normal Sprint 1 position with fifteen days to go, and it's a better thing to say than to be caught out mid-demo.

Message for the group, to land before anyone spends the morning debugging a connection:

"Had a look — I don't think it's a connection problem. There's no deployed URL recorded anywhere: `azure-backend.md` and `azure-fronted.md` both still say 'Backend Azure URL' / 'Frontend Azure URL' as placeholders, and `azure.md` describes the deployment as something to prototype before Milestone 1. So unless someone's deployed it and not written the URL down, there's nothing for the frontend to call. DB is fine, I've been running against it all day. Suggest we demo locally tomorrow and treat the deployment as an open item — @Dean is that right or have I missed something?"

Ends with a question to the person who'd know, which is the right shape when you're telling four people their assumption is wrong.

backend URL: statsthegame-api-dev-eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net

dean send me this

So it is deployed — just not written down anywhere, which is why nobody could find it.

Test it. The health endpoint is the quickest check:

powershell

```
curl.exe -i https://statsthegame-api-dev-eecff5bbfjbyhbb2.southafricanorth-01.azureweb
```

If that path is wrong, try the root and a versioned path:

powershell

```
curl.exe -i https://statsthegame-api-dev-eecff5bbfjbyhbb2.southafricanorth-01.azureweb
```

Paste both. What we learn: whether the backend is up at all, and whether it can reach the database. If health responds but a data endpoint 500s, that's a missing `DATABASE_URL` in App Service configuration — which is a real, fixable thing.

Also worth noting the region: **southafricanorth**. So the backend is in South Africa and the database is in Frankfurt. Every query from the deployed API pays that 185ms round trip, exactly as ADR-003 predicted. Your latency finding applies to production, not just your laptop.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i https://statsthegame-api-dev-eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net/health
```

```
HTTP/1.1 503 Service Unavailable
```

```
Content-Type: text/html; charset=utf-8
```

```
Date: Mon, 10 Aug 2026 20:31:24 GMT
```

```
Transfer-Encoding: chunked
```

```
<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>503
```

```
Service Unavailable</title><style>{*box-sizing:border-box}body{margin:0;font-family:"Segoe UI",Roboto,Arial,sans-serif;background:#fff;color: #1f2937;min-height:100vh;display:flex;align-items:center;justify-
```

```
content:center;padding:32px}.wrap{width:min(680px,100%);display:flex;flex-direction:column}h1{margin:0 0 14px;font-size:38px;line-height:1.1;letter-spacing:-.02em}.lead{margin:0 0 32px;font-size:17px;line-height:1.65;color: #6b7280;max-width:90ch}.lead p{margin:0 0 14px}.lead p:last-child{margin-
```

```
bottom:0}.controls{display:flex;align-items:center;justify-content:space-between;gap:16px;flex-wrap:wrap}.timestamp{margin:0;font-size:13px;color: #9ca3af}.actions{display:flex;flex-wrap:wrap;gap:12px;justify-content:flex-
```

```
end}.btn{border:1px solid #d1d5db;background:#fff;color: #1f2937;border-
```

```

radius:12px;padding:12px 18px;font-size:14px;font-weight:600;cursor:pointer;text-
decoration:none;display:inline-flex;align-items:center;justify-content:center;min-
height:44px;transition:.2s
ease}.btn:hover{transform:translateY(-1px)}.btn.primary{background:#2563eb
;color:#fff;border-color:#2563eb}.troubleshoot{align-self:flex-end;margin-
top:12px;margin-right:4px;font-size:13px;color:#6b7280;text-
decoration:underline;cursor:pointer}.troubleshoot:hover{color:#2563eb}
}@media (max-width:720px){h1{font-size:30px}}@media (prefers-color-
scheme:dark){body{background:#0f172a;color:#e5e7eb}.lead{color:
#9ca3af}.timestamp{color:#6b7280}.btn{background:#1e293b;color:
#e5e7eb;border-color:#334155}.btn.primary{background:#3b82f6
;color:#fff;border-color:#3b82f6}.troubleshoot{color:#94a3b8}
}.troubleshoot:hover{color:#60a5fa}}
```

```

</style></noscript></head><body><main
class="wrap" aria-labelledby="error-title"><h1 id="error-title">503 Service
Unavailable</h1><div class="lead"><p>The server is unable to handle your
request right now. This is usually temporary, so please try again in a few minutes.
</p></div><div class="controls"><p class="timestamp">UTC time: <span
id="utc-time"></span></p><div class="actions"><button class="btn primary"
onclick="location.reload()">Try again</button></div></div><a
class="troubleshoot" href="https://statsthegame-api-dev-
eecff5bbfjbyhbb2.scm.southafricanorth-
01.azurewebsites.net/detectors">Troubleshoot (site owners only)</a></main>
<script>(function(){var t=new Date().toLocaleString(undefined,
{year:"numeric",month:"2-digit",day:"2-digit",hour:"2-digit",minute:"2-
digit",second:"2-
digit",timeZone:"UTC",timeZoneName:"short"}),e=document.getElementById("utc
-time");e&&(e.textContent=t)}());</script></body></html>
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

Show more

503 from Azure's own error page, not from your app. That means the App Service is provisioned but the Node process either isn't running or is crashing on startup — Azure never gets to route the request.

Common causes, roughly in order: the app crashes on boot because a required environment variable is missing (`DATABASE_URL`, or the CA certificate path); the startup command is wrong so nothing listens; the app doesn't bind to the port Azure supplies via `PORT`; or the free tier has gone idle and is failing to warm.

You can narrow it without dashboard access. Try once more — if it's a cold start it may come up:

powershell

```
curl.exe -i -s -o NUL -w "%{http_code}`n" https://statsthegame-api-dev-eecff5bbfjbyhbb
```

Run that two or three times, twenty seconds apart. A 503 that becomes a 200 is cold start. A persistent 503 is a crash loop.

Beyond that you need the Azure logs, which means dashboard access, which you don't have and shouldn't be chasing at 22:30. That link in the error page — the `/detectors` endpoint — is site-owner only, so it's Dean's.

Paste the status codes and I'll tell you which it is. Then it's a message to Dean, not a fix.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i -s -o NUL -w "%
{http_code} n" https://statsthegame-api-dev-
eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net/health
503
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i -s -o NUL -
w "%{http_code} n" https://statsthegame-api-dev-
eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net/health
503
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i -s -o NUL -w "%
{http_code} n" https://statsthegame-api-dev-
eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net/health
503
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i -s -o NUL -
w "%{http_code} n" https://statsthegame-api-dev-
eecff5bbfjbyhbb2.southafricanorth-01.azurewebsites.net/health
503
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Persistent 503 across four attempts — not a cold start. The app isn't running.

That's as far as you can get from outside. The next step is Azure's log stream, which needs dashboard access you don't have.

One thing worth checking locally, because it's the most likely cause and it's a two-minute look: whether the app binds to Azure's supplied port.

powershell

```
Select-String -Path apps\backend\src\index.ts -Pattern "listen|PORT|process.env" -Cont
```

Azure sets `PORT` and expects the app to listen on it. If the code hardcodes 3000 or reads a differently-named variable, the process starts, listens on the wrong port, Azure's front end can't reach it, and you get exactly this — a 503 with no application error, forever.

Paste that. If it's the port, you can tell Dean precisely what to fix rather than "it's broken." If the port handling is right, it's an environment variable or a crash on boot, and only the logs will say which.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
apps\backend\src\index.ts -Pattern "listen|PORT|process.env" -Context 2,2
> apps\backend\src\index.ts:1:import 'dotenv/config';
apps\backend\src\index.ts:2:
```

```

> apps\backend\src\index.ts:3:import { createApp } from './app';
> apps\backend\src\index.ts:4:import { loadEnvironment } from './config/env';
> apps\backend\src\index.ts:5:import { closeDatabasePool } from './database';
apps\backend\src\index.ts:6:
apps\backend\src\index.ts:7:const environment = loadEnvironment();
apps\backend\src\index.ts:8:const app = createApp({ environment });
apps\backend\src\index.ts:9:
> apps\backend\src\index.ts:10:const server = app.listen(environment.PORT, () =>
{
> apps\backend\src\index.ts:11: console.log( Sport Analytics API listening
on port
${environment.PORT} );
apps\backend\src\index.ts:12:});
apps\backend\src\index.ts:13:
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

Show more

Port handling looks right — it reads `environment.PORT`, which presumably comes from `process.env.PORT`. Worth confirming, since `loadEnvironment` may validate and reject:

powershell

```
Get-Content apps\backend\src\config\env.ts
```

Paste it. This is the most likely culprit now: if `loadEnvironment` requires `DATABASE_URL` or any other variable and throws when it's missing, the process dies on line 7 before it ever listens — which produces exactly the 503 you're seeing, with no application logs at all.

That would make the fix "set the missing variables in App Service configuration," which is a five-minute job for whoever has the dashboard.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
apps\backend\src\config\env.ts
import { z } from 'zod';
const environmentSchema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().positive().max(65_535).default(3000),
  CORS_ORIGINS: z.string().default('http://localhost:5173'),
  SUPABASE_URL: z.string().trim().url('Supabase URL must be a valid URL'),
  SUPABASE_PUBLISHABLE_KEY: z.string().trim().min(1, 'Supabase publishable key is
required'),
});
export type Environment = z.infer<typeof environmentSchema>;
export function loadEnvironment(source: NodeJS.ProcessEnv = process.env):
Environment {
  const result = environmentSchema.safeParse(source);
  if (!result.success) {
    const details = result.error.issues
      .map((issue) => `${issue.path.join('.')} || 'environment'`);

```

```
    ${issue.message} )
    .join('; ');
    throw new Error( Invalid environment configuration: ${details} );
  }
  return result.data;
}
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

There it is, almost certainly.

`SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY` are **required with no defaults**. If either is absent, `loadEnvironment()` throws on line 7 of `index.ts`, the process exits before `app.listen()`, Azure has nothing to route to, and you get a permanent 503 with no application error page — exactly what you're seeing.

And the deployment doc almost certainly doesn't set them. Check:

```
powershell

Get-Content docs\deployment\azure-backend.md
```

The excerpt earlier showed `API_VERSION`, `CORS_ALLOWED_ORIGINS`, `LOG_LEVEL` — note that's `CORS_ALLOWED_ORIGINS` while the schema wants `CORS_ORIGINS`. Different name. That one has a default so it won't crash, but it means CORS is misconfigured too.

Paste the doc and I'll give you the exact list of what's missing, which is the useful thing to hand Dean.

Two findings worth recording tomorrow regardless of the fix: the deployment documentation lists variable names that don't match the schema, and no deployed URL is recorded anywhere — which is why four people spent this evening unable to tell whether the thing was even running.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
docs\deployment\azure-backend.md
# Azure Backend Deployment
## Hosting Platform
Azure App Service (Linux)
## Runtime
Node.js 22 LTS
## Resource
App Service
statsthegame-api-dev
## Environment
Development
## Environment Variables
| Variable      | Description    |
| -----      | -
```

```
| NODE_ENV      | production  |
| API_VERSION   | v1          |
| CORS_ALLOWED_ORIGINS | Frontend Azure URL |
| LOG_LEVEL     | info        |
Database credentials are configured through Azure App Service and are never
committed.
## Logging
Application logging uses the Azure File System logger.
## Deployment
Deployments are performed automatically through Gitea Actions.
## Rollback
Azure supports redeploying any previous successful deployment by rerunning the
workflow.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Confirmed. The documented environment variables and the ones the code requires are almost entirely different sets.

The code requires (`apps/backend/src/config/env.ts`):

Variable	Required?	In the doc?
<code>SUPABASE_URL</code>	Yes, no default — throws if missing	No
<code>SUPABASE_PUBLISHABLE_KEY</code>	Yes, no default — throws if missing	No
<code>NODE_ENV</code>	Defaults to development	Yes
<code>PORT</code>	Defaults to 3000	No — Azure supplies it
<code>CORS_ORIGINS</code>	Defaults to localhost:5173	Doc says <code>CORS_ALLOWED_ORIGINS</code>

The doc lists `API_VERSION` and `LOG_LEVEL`, which the schema doesn't read at all.

So if App Service was configured from that document, the two required Supabase variables are missing, `loadEnvironment()` throws, the process dies before listening, and Azure returns a permanent 503. That matches the symptom exactly.

Two further things: `CORS_ALLOWED_ORIGINS` won't crash it but means CORS defaults to localhost, so even once it boots the browser will block the frontend. And the doc says Node 22 while `package.json` may target something else — worth checking, but secondary.

This is a diagnosis, not a fix — setting the variables needs the Azure dashboard.

Want the message to Dean? It's late, but this is the difference between him spending twenty minutes tomorrow morning and the group spending the meeting apologising.

yes send to me

A Full diagnosis B Shorter

Had a look at the API URL you sent — it's returning 503 on every request, four attempts, so it's not a cold start. Azure's own error page, not ours, which means the Node process isn't running at all.

I think I've found why. `apps/backend/src/config/env.ts` validates the environment on startup and two variables are required with no default:

`SUPABASE_URL`
`SUPABASE_PUBLISHABLE_KEY`

If either is missing, `loadEnvironment()` throws on line 7 of `index.ts`, before `app.listen()` ever runs. Process dies, Azure has nothing to route to, permanent 503 with no app logs. Matches the symptom exactly.

Neither is in `docs/deployment/azure-backend.md`, so if App Service was configured from that doc they won't be set.

Couple of other mismatches in the same doc while I was in there:

- It says `CORS_ALLOWED_ORIGINS`, the code reads `CORS_ORIGINS`. Won't crash it (it has a default) but CORS will fall back to `localhost:5173`, so the browser will block the frontend even once it boots.

... "with API versioned LOG LEVEL" which the backend doesn't read at all



One caveat to hold in mind: this is a strong hypothesis, not a confirmed cause. A permanent 503 with no application error is consistent with a startup crash, and the missing required variables are the obvious candidate — but a wrong startup command or a build failure would look identical from outside. The Azure log stream will say definitively, and only Dean can see it.

If he sets the variables and it still 503s, the logs are the next step rather than more guessing.

he said theres no data so fix that up in the message

Untangled misconception about data availability and backend database

Worth separating two things there, because they're not the same problem — and there's a detail I missed first time round.

The 503 has nothing to do with data. The process isn't running, so it would 503 against a full database just the same.

And there is data: four fixtures, 955 deliveries, seeded this afternoon. What I can't tell you is whether the deployed backend points at that same database — `DATABASE_URL` isn't in the env schema at all, so it's read directly by the pool. That means a missing or wrong `DATABASE_URL` won't crash startup, it'll just fail at query time. So it's a third variable that needs setting, and it fails differently from the other two.

A Corrects the data point B Shorter

It's not a data problem — there is data. I seeded 4 fixtures / 955 deliveries into the Supabase DB this afternoon, npm run db:seed loads it and it's repeatable. The 503 would happen against a full database exactly the same way, because the app isn't running at all.

Checked it four times, permanent 503, and it's Azure's error page rather than ours — so the Node process is dying before it can serve anything.

Best guess why: apps/backend/src/config/env.ts validates the environment on startup and two vars are required with no default — SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY. If either is missing, loadEnvironment() throws on line 7 of index.ts before app.listen() runs. Process dies, Azure has nothing to route to, permanent 503 with no app logs. Neither is in azure-backend.md, so if you set App Service up from that doc they won't be there.

Third one to watch: DATABASE_URL isn't in the schema at all, it's read directly by the pool. So it won't crash startup if it's missing — the API will boot fine and then every data request will fail. Different symptom, same evening wasted. Worth setting all three at once.

Also the doc says CORS_ALLOWED_ORIGINS but the code reads CORS_ORIGINS. Won't crash it, but CORS falls back to localhost so the browser blocks the frontend even once it's up.



The `DATABASE_URL` point is the one I'd make sure survives any editing — it's the failure mode that would waste a second evening, because the API would come up looking healthy and then fail on every query.

can i not check for him?

Not from here — App Service configuration isn't visible from outside, and the log stream needs authenticated dashboard access. That `/detectors` link in the 503 page is site-owner only.

The one thing you could try is the SCM endpoint, which sometimes responds differently and occasionally leaks a startup error:

```
powershell
```

```
curl.exe -i -s -o NUL -w "%{http_code}`n" https://statsthegame-api-dev-eecff5bbfjbyhbb
```

Almost certainly a 401. Worth ten seconds to rule out.

Beyond that, if you want to check it yourself the answer is getting Azure access — which is a reasonable ask given you're the one who diagnosed it, but it's a tomorrow conversation, not a 23:00 one.

Honestly: you've done the useful part. You found a permanent 503, established it's a startup crash rather than a cold start or a data problem, and identified three specific environment variables with three distinct failure modes. That's most of the way to fixed. The remaining bit needs a login you don't have.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
docs\deployment\azure-backend.md
# Azure Backend Deployment
## Hosting Platform
Azure App Service (Linux)
## Runtime
Node.js 22 LTS
## Resource
App Service
statsthegame-api-dev
## Environment
Show more
Development
## Environment Variables
| Variable      | Description      |
| ----- | ----- |
| NODE_ENV      | production      |
| API_VERSION    | v1              |
| CORS_ALLOWED_ORIGINS | Frontend Azure URL |
| LOG_LEVEL      | info            |
Database credentials are configured through Azure App Service and are never
committed.
## Logging
Application logging uses the Azure File System logger.
## Deployment
Deployments are performed automatically through Gitea Actions.
## Rollback
Azure supports redeploying any previous successful deployment by rerunning the
workflow.
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -i -s -o NUL -w "%
{http_code}`n" https://statsthegame-api-dev-
eecff5bbfjbyhbb2.scm.southafricanorth-01.azurewebsites.net/
403
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

403 — locked down as expected. That's the end of what's checkable from outside.

One more thing worth looking at, and it's local: **whether a deployment ever actually ran.** The doc says deployments happen automatically through Gitea Actions, but your handover says there are no CI runners. If that's still true, nothing has ever deployed, and the 503 is because the App Service was created and never had code pushed to it — which would make the environment variables a red herring.

```
powershell
```

```
Get-Content .gitea\workflows\ci.yml
```

```
powershell
```

```
ls .gitea\workflows
```

Paste both. If there's no deploy job, or there is one but no runner to execute it, that changes the message to Dean materially — "set these variables" becomes "has this ever deployed?"

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content .gitea\workflows\ci.yml
name: Continuous Integration
on:
  push:
    branches:
      - main
  pull_request:
jobs:
  quality:
    runs-on: ubuntu-latest
    timeout-minutes: 25
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
          POSTGRES_DB: sport_analytics_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd="pg_isready -U test_user -d sport_analytics_test"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=5
    env:
      NODE_ENV: test
      DATABASE_URL_TEST:
        postgresql://test_user:test_password@localhost:5432/sport_analytics_test
      SUPABASE_URL: https://example.invalid
      SUPABASE_PUBLISHABLE_KEY: test-publishable-key
    steps:
      - name: Check out repository
        uses: actions/checkout@v4
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - name: Install dependencies
```

```
run: npm ci
- name: Install Playwright Chromium
  run: npx playwright install --with-deps chromium
- name: Reset and migrate test database
  run: npm run db:test:reset --workspace=@sport-analytics/backend
- name: Seed test database
  run: npm run db:test:seed --workspace=@sport-analytics/backend
- name: Run repository checks
  run: npm run check
- name: Run database integration tests
  run: npm run test:database
- name: Run browser and accessibility tests
  run: npm run test:e2e
- name: Generate coverage
  run: npm run test:coverage
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

That's a well-built CI workflow — and it contains **no deploy job at all**. Quality checks only: install, migrate a Postgres service container, run checks, tests, e2e, coverage. Nothing that pushes to Azure.

So `azure-backend.md` saying "Deployments are performed automatically through Gitea Actions" is not true of this repository. Either deployment happens some other way — Azure's own GitHub/local git integration, or a manual push — or the App Service was created and never received code.

That reframes the message to Dean. The environment variables might still be wrong, but the prior question is whether anything has ever been deployed to that App Service, and by what mechanism.

Two more things from that file worth noting:

CI pins Node 20. Dean is on 24, you're on 25. Three environments, and CI matches none of them — which is exactly the divergence that produced today's localStorage failures.

There's a `db:test:reset` and `db:test:seed`, and a Postgres service container. So a test database *does* exist in CI. That partly reopens the #63 question — the reference-fixture test could run in CI even if it can't run locally. Worth revisiting when you pick that up, though it needs `db:test:seed` to load your reference fixture.

Amend the message to Dean's opening:

"Actually before the env vars — has this ever deployed? ci.yml has no deploy job at all, it's quality checks only. azure-backend.md says deployments go through Gitea Actions but there's nothing in the workflow that pushes to Azure. So either it's deploying some other way or that App Service has never had code on it, which would explain the 503 on its own."

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> ls .gitea\workflows
Directory: C:\Users\BenSwartz\Sport-Analytics-Tool\.gitea\workflows
Mode                LastWriteTime         Length Name
----                -
-a----          07/08/2026  12:34           1677 ci.yml
-a----          06/08/2026  16:53           889 deploy-backend.yml
-a----          06/08/2026  16:53           909 deploy-frontend.yml
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

There are deploy workflows — I was wrong to conclude from `ci.yml` alone.

powershell

```
Get-Content .gitea\workflows\deploy-backend.yml
```

Paste it. What I want to see: what triggers it, whether it sets environment variables, and whether it would have run given there are no CI runners.

That last point still stands and may be decisive — a workflow that exists but has never executed produces exactly the same result as no workflow at all.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
.gitea\workflows\deploy-backend.yml
name: Deploy backend
on:
  push:
    branches:
      - main
  paths:
    - 'backend/**'
    - '.gitea/workflows/deploy-backend.yml'
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4
      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - name: Install dependencies
        run: npm ci
      - name: Check backend
        run: npm run check --workspace=backend
      - name: Test backend
        run: npm test --workspace=backend
      - name: Build backend
        run: npm run build --workspace=backend

```

```
- name: Deploy backend
uses: Azure/webapps-deploy@v3
with:
  app-name: statsthegame-api-dev
  publish-profile: ${ secrets.AZURE_BACKEND_PUBLISH_PROFILE }
  package: backend
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Identified workflow configuration mismatches in path filtering

That workflow has almost certainly never run successfully, and there are three independent reasons.

The path filter never matches. It triggers on `backend/**`, but the backend lives at `apps/backend/**`. No push to the backend has ever triggered this workflow. Only a change to the workflow file itself would fire it.

The workspace name is wrong. `npm run check --workspace=backend` — the workspace is `@sport-analytics/backend`. Those three steps would fail immediately if it ever did run.

The package path is wrong. `package: backend` points at a directory that doesn't exist, so the Azure deploy step would upload nothing.

Same is likely true of `deploy-frontend.yml` — worth checking:

```
powershell
```

```
Get-Content .\gitea\workflows\deploy-frontend.yml
```

So the picture is: the App Service exists, the deploy workflow exists, and the two have never met. The 503 is because there's no application deployed, not because of environment variables. Those are still wrong and will bite next, but they're the second problem, not the first.

Dated 06/08 — written two days after Dean's monorepo restructure, against the pre-restructure paths.

This also explains why nobody noticed: with no CI runners, the workflow never ran, so it never failed visibly. A broken workflow that never executes looks identical to a working one.

Paste the frontend workflow and I'll rewrite the message.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
.gitea\workflows\deploy-frontend.yml
name: Deploy frontend
on:
  push:
    branches:
```

```
- main
paths:
  - 'frontend/**'
  - '.gitea/workflows/deploy-frontend.yml'
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4
      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - name: Install dependencies
        run: npm ci
      - name: Check frontend
        run: npm run check --workspace=frontend
      - name: Build frontend
        run: npm run build --workspace=frontend
      env:
        VITE_API_BASE_URL: ${ secrets.VITE_API_BASE_URL }
      - name: Deploy frontend
        uses: Azure/webapps-deploy@v3
        with:
          app-name: statsthegame-web-dev
          publish-profile: ${ secrets.AZURE_FRONTEND_PUBLISH_PROFILE }
          package: frontend/dist
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Identical set of faults: `frontend/**` instead of `apps/frontend/**`, `--workspace=frontend` instead of `@sport-analytics/frontend`, `package: frontend/dist` instead of `apps/frontend/dist`.

Both deploy workflows were written on 6 August against paths that don't exist in the monorepo. Neither has ever run.

A Full — corrects earlier message B Shorter

Dug further and I think I had it wrong — it's not the env vars, or not only. I don't think anything has ever deployed.

Both deploy workflows trigger on the wrong paths:

deploy-backend.yml watches 'backend/**' but the backend is at 'apps/backend/**'

deploy-frontend.yml watches 'frontend/**' but it's at 'apps/frontend/**'

So no push to either app has ever triggered them. Only editing the workflow files themselves would fire them.

And if they did fire they'd fail anyway — they run `npm run check --workspace=backend`, but the workspace is `@sport-analytics/backend`. Same for frontend. And the Azure deploy step uploads package: `backend / frontend/dist`, neither of which exists.

They're dated 06/08, so they were written against the old layout before the monorepo restructure and never updated.

The reason nobody caught it is we've got no CI runners — the workflows never ran, so they never visibly failed. A broken workflow that never executes looks exactly like a working one.



One caveat worth holding: I'm inferring that nothing has deployed from the workflow being incapable of deploying. It's possible Dean deployed manually at some point, via VS Code or the Azure CLI, in which case something *is* on there and the env vars are back in play. His answer will settle it.

Either way, the workflows need fixing before deployment can be reliable — and with no runners, they still won't execute even once corrected. That's the deeper problem, and it's a Sprint 1 rubric item: CI/CD is a named key requirement in the brief.

_chat.txt

TXT

did he not give us credentials for access?

Examined timeline to clarify credential sharing versus infrastructure readiness

No — he shared the URL, not credentials. And the chat turns up something that changes the picture.

Dean posted that backend URL on 5 August at 21:23, in the group. Then at 22:06: *"ive done the azure"*, immediately followed by *"it wont work cause ci/cd is not workkiing"* and *"but its there for when brendan and pravesh sort it"*.

So the 503 is known and expected. Shayna had documented the same thing earlier that day: CI can't work until Pravesh provisions runners, checks will sit yellow, merge anyway.

That means tonight's diagnosis isn't news to Dean — he already knew nothing was deployed. What is news is the part he doesn't know: **the deploy workflows are broken independently of the runners**. Everyone is waiting on Pravesh in the belief that deployment starts working once runners exist. It won't. `deploy-backend.yml` watches `backend/**` and the app is at `apps/backend/**`, so the workflow won't even trigger; and if it did, `--workspace=backend` and `package: backend` are both wrong. Same three faults in the frontend one.

That's worth sending tonight, because it changes what "waiting on Pravesh" means.

Two side notes from the export:

Nadav asked for Supabase access on 5 August to the same personal email, and again tonight. Nobody has actioned it in five days.

A live Supabase service-role secret is sitting in this chat from 15 April —

`SUPABASE_SERVICE_ROLE_KEY = sb_secret_...`, pasted in plain text. Previous project, but a service-role key bypasses all row-level security. If that project's Supabase instance still exists, that key should be rotated.

how can i test if the website works locally on my machine, to show the "client"

Locally is entirely doable — the database is up, the seed is loaded, and the API and frontend both merged today.

Step 1 — check what dev scripts exist and what the frontend needs:

powershell

```
Select-String -Path package.json -Pattern "dev|start"
```

powershell

```
Select-String -Path apps\frontend\package.json -Pattern "dev|scripts" -Context 0,6
```

powershell

```
Select-String -Path apps\frontend\src\api\client.ts -Pattern "VITE_|BASE_URL|baseUrl"
```

Paste all three. I need to know how the frontend is told where the API lives before telling you what to put in your env files — and whether there's a root `dev` script that runs both together.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path package.json -
Pattern "dev|start"
package.json:14: "dev": "npm run dev --workspace=@sport-analytics/frontend",
package.json:15: "dev:frontend": "npm run dev --workspace=@sport-
analytics/frontend",
package.json:16: "dev:backend": "npm run dev --workspace=@sport-
analytics/backend",
package.json:35: "devDependencies": {
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
apps\frontend\package.json -Pattern "dev|scripts" -Context 0,6
> apps\frontend\package.json:6: "scripts": {
> apps\frontend\package.json:7: "dev": "vite",
apps\frontend\package.json:8: "build": "tsc -b && vite build",
apps\frontend\package.json:9: "preview": "vite preview",
apps\frontend\package.json:10: "lint": "eslint src --ext .ts,.tsx",
apps\frontend\package.json:11: "typecheck": "tsc --noEmit",
apps\frontend\package.json:12: "test": "vitest run"
apps\frontend\package.json:13: },
> apps\frontend\package.json:21: "devDependencies": {
apps\frontend\package.json:22: "@testing-library/jest-dom": "^6.6.3",
apps\frontend\package.json:23: "@testing-library/react": "^16.0.1",
apps\frontend\package.json:24: "@types/react": "^18.3.12",
apps\frontend\package.json:25: "@types/react-dom": "^18.3.1",
apps\frontend\package.json:26: "@vitejs/plugin-react": "^4.3.3",
apps\frontend\package.json:27: "jsdom": "^25.0.1",
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
apps\frontend\src\api\client.ts -Pattern "VITE_|BASE_URL|baseUrl" -Context 1,1
apps\frontend\src\api\client.ts:6:
> apps\frontend\src\api\client.ts:7:const apiBaseUrl =
import.meta.env.VITE_API_BASE_URL ??
'http://localhost:3000/api/v1';
apps\frontend\src\api\client.ts:8:
apps\frontend\src\api\client.ts:50: const normalizedPath = path.startsWith('/')
? path
: `/${path}`;
> apps\frontend\src\api\client.ts:51: const response = await
fetch( `${apiBaseUrl}${normalizedPath}`, {
apps\frontend\src\api\client.ts:52:   ...init,
apps\frontend\src\api\client.ts:69:export async function getHealth(signal?:
AbortSignal):
Promise<HealthResponse> {
> apps\frontend\src\api\client.ts:70: const response = await
fetch( `${apiBaseUrl}/health`, {
apps\frontend\src\api\client.ts:71:   headers: {
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

```

The frontend defaults to `http://localhost:3000/api/v1`, which is exactly where a local backend will be. So no env var needed on the frontend side if you run the backend on its default port.

Step 1 — start the backend in one terminal:

```
powershell  
npm run dev:backend
```

It needs `SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY` in `apps/backend/.env` or it'll throw on startup — same failure as Azure. If it does, paste the error.

Step 2 — start the frontend in a *second* terminal:

```
powershell  
cd C:\Users\BenSwartz\Sport-Analytics-Tool  
npm run dev:frontend
```

Vite will print a localhost URL, usually `http://localhost:5173`.

Step 3 — check the API directly first, so you know which half is broken if something fails:

```
powershell  
curl.exe -s http://localhost:3000/api/v1/health
```

Paste the backend startup output and that curl result. Once the API answers, open the Vite URL and see whether the browse pages show your four fixtures.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run dev:backend  
> sport-analytics-tool@0.1.0 dev:backend  
> npm run dev --workspace=@sport-analytics/backend  
> @sport-analytics/backend@0.1.0 dev  
> tsx watch src/index.ts  
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\src\config\env.ts:21  
  throw new Error( Invalid environment configuration: ${details} );  
    ^  
Error: Invalid environment configuration: SUPABASE_URL: Required;  
SUPABASE_PUBLISHABLE_KEY: Required  
    at loadEnvironment (C:\Users\BenSwartz\Sport-Analytics-  
Tool\apps\backend\src\config\env.ts:21:11)  
    at <anonymous> (C:\Users\BenSwartz\Sport-Analytics-  
Tool\apps\backend\src\index.ts:7:21)  
    at Object.<anonymous> (C:\Users\BenSwartz\Sport-Analytics-  
Tool\apps\backend\src\index.ts:59:2)  
    at Module._compile (node:internal/modules/cjs/loader:1829:14)  
    at Object.transformer (C:\Users\BenSwartz\Sport-Analytics-  
Tool\node_modules\tsx\dist\register-BcxQ17Jo.cjs:9:3339)  
    at Module.load (node:internal/modules/cjs/loader:1552:32)  
    at Module._load (node:internal/modules/cjs/loader:1354:12)  
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)  
    at loadCJSModuleWithModuleLoad  
(node:internal/modules/esm/translators:327:3)  
    at ModuleWrap.<anonymous> (node:internal/modules/esm/translators:232:7)  
Node.js v25.9.0
```

Show more

There's your confirmation — the exact same error that's killing it on Azure, reproduced locally. That's the diagnosis proven rather than inferred, and worth telling Dean: it's not a guess any more.

Your `apps/backend/.env` has `DATABASE_URL` but not the two Supabase variables, because you added it for the database work before the auth code landed.

Step 1 — see what your .env has and what the example says it should:

```
powershell
```

```
Get-Content apps\backend\.env.example
```

```
powershell
```

```
(Get-Content apps\backend\.env) -replace '=.*', '<redacted>'
```

The second redacts values so nothing sensitive lands in this chat.

Paste both. You need real values for `SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY` — the URL is the project URL from the Supabase dashboard, and the publishable key is the public anon key, which is safe to hold locally but shouldn't be pasted here. Gabriel set up the auth, so he'll have both, or they're in the Supabase project settings if you have dashboard access.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
apps\backend\.env.example
NODE_ENV=development
PORT=3000
CORS_ORIGINS=http://localhost:5173
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/sport_analytics
# Public Supabase project configuration used for authentication.
SUPABASE_URL=https://your-project-ref.supabase.co
SUPABASE_PUBLISHABLE_KEY=your-supabase-publishable-key
# Add only server-side external API credentials here.
EXTERNAL_API_KEY=
API_VERSION=v1
CORS_ALLOWED_ORIGINS=http://localhost:5173
LOG_LEVEL=debug
# Dedicated isolated test database.
# This must never point to the development or production database.
DATABASE_URL_TEST=postgresql://postgres:postgres@localhost:5432/sport_analytics_test
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

The example has both variables, so your `.env` predates them. You need the two real values.

Where to get them: both are in the Supabase dashboard under Project Settings → API — the project URL and the publishable/anon key. If you have dashboard access, two minutes. If not, Gabriel has them from the auth work, or they're in his `.env`. Both are public-facing values by design, so asking for them in the group is fine, unlike a service-role key.

Then add to `apps/backend/.env`:

```
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_PUBLISHABLE_KEY=<the anon key>
```

Then retry:

```
powershell
npm run dev:backend
```

Worth noting for the Azure message: this also tells you the App Service almost certainly has the same gap. The `.env.example` was updated when auth landed, but nobody revisited the Azure configuration — so even once the deploy workflows are fixed, the app will still crash on startup unless those two are added there too. Three problems stacked: workflows never trigger, no runners to run them, and missing env vars at the end of it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run dev:backend
> sport-analytics-tool@0.1.0 dev:backend
> npm run dev --workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 dev
> tsx watch src/index.ts
Sport Analytics API listening on port 3000
```

Backend is up. Now check it can actually reach the database — startup succeeding only proves the env parsed, not that queries work.

In a second terminal:

```
powershell
curl.exe -s http://localhost:3000/api/v1/health
```

```
powershell
curl.exe -s http://localhost:3000/api/v1/competitions
```

Paste both. Health should respond; competitions should return your seeded data — IPL, the Africa Continental Cup, the Asian Games, and whatever 423788's competition is.

If competitions returns data, the API works end to end and you can start the frontend.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -s
http://localhost:3000/api/v1/health
{"status":"ok","service":"sport-analytics-api","timestamp":"2026-08-
10T20:47:05.727Z"}
PS C:\Users\BenSwartz\Sport-Analytics-Tool> curl.exe -s
http://localhost:3000/api/v1/competitions
{"data":[{"competitionId":"2","name":"Africa Continental Cup"},
{"competitionId":"1","name":"Asian Games Men's Cricket Competition"},
{"competitionId":"3","name":"Australia in New Zealand T20I Series"},
{"competitionId":"4","name":"Indian Premier League"}],"pagination":
{"nextCursor":null}}
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

End to end. Health responds, and competitions returns all four seeded competitions with pagination — that's the API reading your data from Supabase.

Start the frontend in the second terminal:

```
powershell
npm run dev:frontend
```

Then open the URL Vite prints — usually `http://localhost:5173`. Browse to the competitions or fixtures pages and you should see this same data rendered.

If anything fails there, open devtools Network tab and tell me the failing request. Since the backend is confirmed working, any failure from here is frontend-side — CORS, a wrong path, or a rendering issue.

Once it works: that's your demo for tomorrow. Two terminals, local, real data, four fixtures chosen specifically because they break naive models. And you now know exactly why the deployed version doesn't work, which is a better answer than not knowing.

So what must I do?
meanwhile, got this message from dean

The message from Dean didn't come through — nothing after "meanwhile, got this message from dean". Paste it and I'll factor it in.

Meanwhile, what to do right now is one command:

```
powershell
npm run dev:frontend
```

in a second terminal, then open the URL it prints. That's the last step to having a working local demo. Everything below it is confirmed working.

why is the screen blank?

A blank page usually means a JavaScript error before React mounts, and the browser console will name it.

Open devtools (F12), Console tab, reload. Paste the first error.

Also paste what the Vite terminal printed — the URL it's serving on, and any errors after startup.

Most likely candidate given what we've seen: the frontend has its own required env vars for Supabase auth. `AuthProvider` and `supabase-client.ts` landed today, and if `supabase-client.ts` calls `createClient` with `undefined`, it throws at import time and nothing renders. Check:

```
powershell
```

```
Get-Content apps\frontend\src\features\auth\supabase-client.ts
```

```
powershell
```

```
Get-Content apps\frontend\.env.example
```

Paste those two as well — likely the same missing-variable problem one layer up.

f12 make me take a screenshot

F12 is bound to screenshot on your machine. Alternatives for opening devtools in Chrome or Edge:

- **Ctrl+Shift+J** — opens straight to the Console
- **Ctrl+Shift+I** — opens devtools generally
- Or right-click the blank page → **Inspect**, then click the Console tab

Paste the first red error you see.

While you're doing that, these two will likely tell us the answer anyway:

```
powershell
```

```
Get-Content apps\frontend\src\features\auth\supabase-client.ts
Get-Content apps\frontend\.env.example
```

Uncaught Error: VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY are required for frontend authentication.
at supabase-client.ts:7:9

Same problem, one layer up. The frontend needs its own copies with the `VITE_` prefix.

Create `apps/frontend/.env` with the same two values you just put in the backend:

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<the anon key>
```

Same values as `SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY` in `apps/backend/.env`, just renamed — Vite only exposes variables beginning with `VITE_` to browser code.

Check what else the example expects first, in case there's a third:

```
powershell
Get-Content apps\frontend\.env.example
```

Then restart the frontend — Vite doesn't pick up new env files without a restart. Ctrl+C in that terminal, then `npm run dev:frontend` again.

Worth noting for tomorrow: the deployed frontend will need these too, set at build time. So that's four environment variables across two App Services, none of them documented in the Azure deployment pages.

it works!

Working end to end, locally: Supabase → backend API → React frontend, with real ball-by-ball data behind it.

Two things worth doing before you sleep, both short.

Write down what you did. Four environment variables across two apps, and none of them are in the setup documentation — otherwise tomorrow morning four people repeat tonight independently. Even a WhatsApp message with the variable names is enough; the doc can follow.

Tell the group it runs. Right now Shayna, Nadav and Dean all think nothing works. It does — just not deployed.

Something like:

"Got it running locally, works end to end — real data from the DB through the API into the frontend. The trick is you need 4 env vars nobody's documented: apps/backend/.env needs SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY, and apps/frontend/.env needs the same two values as VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY. Then npm run dev:backend in one terminal and npm run dev:frontend in another. That's also exactly why Azure 503s — same missing vars, plus the deploy workflows have never actually run. So we can demo locally tomorrow, we just can't show a deployed URL."

That converts your evening into something the whole group can use, and it means the client meeting has a working demo instead of an apology.

"So what must I do?"
from dean

Delineated Dean's actionable tasks versus dashboard-only configurations

A Ordered, with the shortcut B Shorter

Three things, and they're independent — fixing one doesn't fix the others.

****1. Env vars in App Service (only you can do this)****

Backend needs SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY. I proved this is real — ran the backend locally without them and got the identical crash, added them and it booted fine. Also needs DATABASE_URL (not in the schema so it won't crash, it'll just fail every query) and CORS_ORIGINS — note the doc says CORS_ALLOWED_ORIGINS which the code doesn't read.

Frontend needs VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY at build time, plus VITE_API_BASE_URL.

****2. The deploy workflows are broken (I can fix this in a PR)****

Both watch the wrong paths — backend/** and frontend/** instead of apps/backend/** and apps/frontend/**. So they've never triggered. And --workspace=backend should be @sport-analytics/backend, and the package paths don't exist. Written 06/08 against the old layout.

****3. No runners — blocked on Pravesh****

Even with 1 and 2 fixed, nothing runs automatically.

So if you want a working URL without waiting for Pravesh, the fastest path is deploying manually from your machine — Azure CLI or the VS Code Azure extension — which skips Clientify. Set the env vars first or



The manual-deploy suggestion is the one that might actually get you a live URL before the meeting — it sidesteps the runner problem entirely, and Dean already has the Azure access to do it.

One honest caveat if he tries it: I don't know how the App Service is configured to start a Node app from a monorepo, and `apps/backend` needs building and its workspace dependencies resolving. A manual deploy may need a bit of fiddling with the startup command. Worth him trying, not worth promising.

[22:53, 10/08/2026] Ben Swartz: again, from claude

[22:54, 10/08/2026] Dean Feldman: but is the azure the reason localhost not working with DB?

[22:54, 10/08/2026] Ben Swartz: No, people just don't have vars set up

[22:55, 10/08/2026] Dean Feldman: ive got them set up

[22:55, 10/08/2026] Dean Feldman: what is your env

[22:55, 10/08/2026] Dean Feldman: i sent mine to the group yesterday

Identified security risks and mapped environment variable troubleshooting strategy

A Helpful + raises the secret issue

B Shorter

Names rather than values, since these shouldn't really live in WhatsApp:

`apps/backend/.env`

`NODE_ENV`, `PORT`, `CORS_ORIGINS`, `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`

`apps/frontend/.env`

`VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`

(same two values as the backend ones, just renamed — Vite only exposes `VITE_` prefixed vars to the browser)

Then `npm run dev:backend` in one terminal, `npm run dev:frontend` in another.

If you've got all six and it's still not working, what's the actual error? The failure modes are pretty distinct:

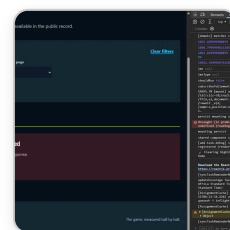
- backend crashes on startup with "Invalid environment configuration" → missing `SUPABASE_URL` or `SUPABASE_PUBLISHABLE_KEY`
- backend starts, `curl http://localhost:3000/api/v1/health` works, but `/competitions` fails → `DATABASE_URL` wrong
- blank white page in the browser → missing the `VITE_` ones (that was my problem, took me ages)

Worth checking the health and competitions endpoints with curl before opening the browser, tells you



The secret point is worth making even though it's late and it's the old project. A service-role key in a WhatsApp group is the kind of thing that reads badly if anyone ever looks, and the fix is one click in the Supabase dashboard.

Separately: if Dean has all six set and it still fails, his error message will tell you which of the three it is in about ten seconds. That's the useful part of the reply.



dean gets this

Isolated browser extension noise masking actual API error

A Full diagnosis B Shorter

Good news in that screenshot actually — the page renders, so your VITE_ vars are fine. It's just the API call failing.

Two things:

That console is useless for this, it's drowning in browser extension noise — bettercampus, AssignmentCache, LMS adapter, all of that is an extension not our app. Even the red "cannot read properties of undefined (reading 'query')" is at all.js:606 which is the extension. Try an incognito window with extensions off, or just use the **Network** tab instead of Console — click the failing request and look at the status code and response.

But faster: skip the browser entirely and hit the API directly.

```
curl http://localhost:3000/api/v1/health
curl http://localhost:3000/api/v1/competitions
```

health fails or nothing responds → backend isn't running, check that terminal
health works but competitions errors → DATABASE_URL is wrong or pointing at an empty DB
both work in curl but the browser still fails → CORS, check CORS_ORIGINS is http://localhost:5173 (note CORS_ORIGINS not CORS_ALLOWED_ORIGINS — the code only reads the first one, the doc has it wrong)



My guess is his backend isn't running at all — the frontend renders fine standalone, and it's easy to start only `dev:frontend` and forget the second terminal. The curl on `/health` settles it in one command.

for the database size issue, could we not have multiple databases?

Weighed multi-database workaround against architectural constraints and terms violations

Technically yes, but I'd argue against it — and there's a better answer underneath the question.

Why sharding across databases is bad here: Postgres can't join across separate instances without foreign data wrappers, and your entire model is joins — delivery → innings → fixture → competition. Split by competition and single-fixture queries still work, but career aggregates break, because a player's career crosses competitions. That's the thing the platform exists to do. You'd also be running migrations N times, unable to wrap a submission in one transaction, and explaining a sharded free-tier arrangement to a marker who asked for a coherent platform. Supabase also caps free projects per organisation, so you'd be creating orgs to dodge a limit, which isn't a defensible thing to write up.

The legitimate version of the idea is splitting by data type rather than by shard: source JSON files and published dataset releases go to object storage, only normalised event data stays in Postgres. Your handover already names that as an untested option. It's real and it's clean.

But the more important thing: re-read the project brief. The intermediate tier says the platform should be exercised at *"hundreds of fixtures, each with hundreds of events."* Hundreds. You're planning for 13,953 matches and 3.2 million deliveries because ADR-003 agreed a scope of eleven franchise competitions plus men's and women's internationals — but nothing in the brief asks for that.

At 500 MB you can hold roughly 1,000 matches. That is comfortably above what the brief requires and would still be a genuinely large dataset to demo. The storage problem may be entirely self-inflicted by a scope decision made on 5 August that nobody has revisited against what's actually being marked.

That's the thing to put to the group: not "how do we fit 3.2 million deliveries into 500 MB", but "why did we commit to 3.2 million deliveries when the brief asks for hundreds of fixtures?"

Spoke to my lecturer, he said we either need to pay or find a new hosting service that's big enough - do the research for me

Scouting affordable Postgres hosting options meeting your specifications

Worth doing properly — let me check current pricing and options rather than rely on memory.

Excavated Azure Students option and compared database hosting alternatives comprehensively

Researched. The short answer: **you probably don't need to pay at all**, and the best option also fixes the latency problem.

Created a file, read a file

The headline: an Azure free account gives you PostgreSQL Flexible Server free for twelve months — 750 hours of B1MS, enough to run continuously, plus 32 GB storage and 32 GB

backup, and the Azure for Students page lists the same allowance. That's sixty-four times your current limit, at no cost, on the platform Dean has already set up.

And because you'd put it in South Africa North alongside the App Service, it also kills the 181 ms latency. Two problems, one move, zero rand.

Three caveats worth carrying into the conversation. I haven't verified the offer applies to the subscription Dean created or that it's available in South Africa North — both need checking in the portal before anyone commits. Supabase Auth would stay, so you'd depend on two providers instead of one. And the twelve months runs from account creation, which covers the course but not forever.

Worth telling your lecturer that his framing may be slightly off: it isn't pay-or-move, it's move-and-pay-nothing.

The one thing I'd raise separately — the brief asks for the platform to handle *hundreds* of fixtures with hundreds of events each. Your 13,953-match scope was a group decision on 5 August, not a requirement. Even at 500 MB you could hold roughly a thousand matches. The storage problem may be partly self-inflicted, and that's worth five minutes before anyone migrates anything.



Database hosting options

Document · MD



Download

how much work is it migrate to azure?

Evaluated migration complexity across infrastructure, certificates, and configuration updates

Less than you'd think, and the reason is that **there's no data to migrate**. Four fixtures, all reproducible from files committed in the repo. It's a re-point, not a migration.

What's actually involved:

Task	Who	Time
Create Flexible Server in South Africa North	Dean	15 min
Firewall rules for six members plus Azure services	Dean	10 min
Swap the TLS certificate handling	You	30-60 min
Update DATABASE_URL in six .env files and App Service	Everyone	5 min each
db:migrate, db:seed, db:validate	You	15 min
Rewrite ADR-003 or write ADR-005	You	45 min

Task	Who	Time
Update <code>setup.md</code> , <code>overview.md</code> , <code>repository-structure.md</code>	You	45 min

Call it **half a day of real work**, most of it documentation rather than code.

The one fiddly part is TLS. You currently pin a committed Supabase CA at `apps/backend/certs/supabase-ca.crt`, and several files hardcode that path — `ingest-match.ts`, `sql.ts`, `latency.ts`, `validate-match.ts`, the `db:migrate` script via `NODE_EXTRA_CA_CERTS`, and probably `src/database/pool.ts`. Azure's certificates chain to a well-known root, so `rejectUnauthorized: true` with no custom CA may just work — which would simplify the code rather than complicate it. But that needs testing, not assuming.

Check the blast radius before committing to an estimate:

```
powershell

Select-String -Path apps\backend\**\*.ts,apps\backend\package.json -Pattern "supabase-
```

Two things that make it cheaper than it looks: nothing in the application uses a provider-specific interface, so no application code changes; and your migration has been rolled back and reapplied cleanly several times, so you know it works from empty.

One thing that makes it more expensive: you'd be doing it while five other people are actively developing against the current database. Everyone changes `.env` on the same day or half the team is broken. That's a coordination cost, not a technical one — best done at a natural break, and announced rather than sprung.

dean said its 600\$
give him step by step instructions on how ot launch it

Diagnosed pricing discrepancy and mapped corrective portal navigation steps

\$600 means he's pricing the wrong thing. The Azure portal defaults to a General Purpose instance — typically D2ds_v5 with large storage — which is roughly that per year. The free offer is a specific configuration you have to select deliberately: **Burstable, Standard_B1ms, 32 GiB**. Anything else and it bills.

There's also a prior question: the free offer applies to an **Azure free account or Azure for Students** subscription within 12 months of creation. If Dean's subscription is pay-as-you-go, it doesn't apply at all and \$600 is real. That's the first thing to check.

Steps for him:

0. Check the subscription is eligible. Azure portal → search **Free Services**. If Azure Database for PostgreSQL appears as a tile there, the offer applies. If the page says no free offers

available, stop — the subscription isn't eligible and we need a different plan.

1. Search **Azure Database for PostgreSQL flexible servers** → **Create** → **Flexible server**.

2. Basics tab:

- Subscription: the free/student one, not a pay-as-you-go
- Resource group: same one as the App Service
- Server name: `statsthegame-db-dev`
- Region: **South Africa North** (this is what kills our 185ms latency)
- PostgreSQL version: **16**
- Workload type: **Development**
- High availability: **disabled** (it doubles the cost)

3. **Compute + storage** — **this is where the \$600 comes from**. Click **Configure server** and set:

- Compute tier: **Burstable**
- Compute size: **Standard_B1ms** (1 vCore, 2 GiB RAM)
- Storage: **32 GiB**
- Storage auto-growth: **disabled** (or it silently grows past free)
- Backup retention: **7 days**, geo-redundant **off**

The estimated monthly cost shown on that page should drop to roughly zero. **If it doesn't say ~\$0, don't create it** — send me a screenshot instead.

4. **Authentication**: PostgreSQL authentication only. Set an admin username and a strong password — into the group's password manager or a DM, not the group chat.

5. **Networking**: Public access. Then add firewall rules:

- Tick **Allow public access from any Azure service within Azure** (so the App Service can reach it)
- Add each member's current IP — the portal has an "Add current client IP" button, we'll each need our own added

6. **Review + create**. Takes about 5 minutes to provision.

7. **Send me the connection string** (Settings → Connect) and I'll run the migrations and seed, and check TLS.

One thing to flag: 32 GiB storage on the free offer, but **storage can only ever be scaled up, never down** on Flexible Server. Set it to 32 and leave it.

If step 0 shows the subscription isn't eligible, tell me before doing anything else — the whole plan changes and Supabase Pro at \$25/month becomes the answer.

@Ben Swartz what do you think about this
<https://www.oracle.com/database/free/get-started/>
20GB free, its not postgre but shouldnt be an issue

dean sent me this

Identified fundamental incompatibilities between PostgreSQL schema and Oracle database

Let me check what that page actually offers before answering — Oracle has several things called "free" and they're very different.

Weighed Oracle migration complexity against apparent simplicity

Checked it — that page isn't a hosted database at all. It's a **download**: RPMs for Oracle Linux, a Windows zip, a Docker image. You'd have to host it yourself on a VM, which is more infrastructure work, not less. The 20 GB he's thinking of is probably OCI's Always Free Autonomous Database, which is a different product on a different page.

But the bigger issue is "it's not Postgres but shouldn't be an issue" — it would be a substantial issue, and not only for your work.

A Full technical case B Shorter

Two problems with that one unfortunately.

First, that page isn't a hosted database — it's a download. RPMs for Oracle Linux, a Windows zip, a Docker image. We'd have to run it on a VM ourselves and manage it. The 20GB you're thinking of is probably OCI's Always Free Autonomous DB, which is a different product.

Second, and this is the real one — "not Postgres shouldn't be an issue" would actually be a big issue. Our whole stack is Postgres-specific:

- The migration uses ENUM types, text[] arrays, jsonb, timestamptz, partial unique indexes (WHERE superseded_at IS NULL), UNIQUE NULLS NOT DISTINCT, num_nonnulls() — Oracle has none of those, or has different versions
- ON CONFLICT DO NOTHING doesn't exist, that's a MERGE statement in Oracle. That's what makes our ingestion idempotent
- COUNT(*) FILTER (WHERE ...) isn't supported — that's in most of our derivation queries
- node-pg-migrate is Postgres-only, so all the migration tooling goes
- the pg driver goes, so src/database/pool.ts, query.ts, transaction.ts all get rewritten

And it's not just my stuff — Shayna's repositories and the public-read service that merged yesterday are all PostgreSQL. That's another 1000 lines



Ending on the Free Services check is the useful part — it redirects him from finding a new provider back to the one question that actually resolves this. If PostgreSQL shows up as a tile, the \$600 was a misconfiguration and you're done.

One thing worth keeping in mind on the scope question, which nobody has revisited: the brief asks for hundreds of fixtures with hundreds of events each. Even 500 MB holds roughly a thousand matches. If the Azure free offer turns out not to apply, "load fewer matches" is a legitimate answer rather than a defeat — and it costs nothing at all.

he said he keeps getting charged for everything?
can you please look into OceanDB

"OceanDB" isn't something I recognise as a hosted Postgres provider — I suspect he means **DigitalOcean Managed Databases**, which is a real and reasonable option. Let me check rather than assume.

Searched the web

DigitalOcean is real Postgres — no rewrite, unlike the Oracle idea — but it has **no free tier**. Cheapest managed Postgres is \$15.15/month, and new accounts get \$200 credit valid for 60 days. So it's free until roughly mid-October, then \$15/month, and the credit expires right around your final submission.

Compared with what you already have on the table:

Option	Storage	Cost to 11 Oct	After
Azure Flexible Server (free offer)	32 GB	\$0	\$0 for 12 months
DigitalOcean	~10 GB at base plan	\$0 (credit)	\$15/mo
Supabase Pro	8 GB	~\$50	\$25/mo
Supabase free (current)	500 MB	\$0	\$0

DigitalOcean is a perfectly decent fallback and it's genuinely free for the project's duration. It doesn't fix the 185 ms latency though — no South African region — and it's another provider to manage.

But Dean saying "he keeps getting charged for everything" is the thing to sort out, not route around. Azure charges for what you provision. If his subscription is pay-as-you-go rather than Azure for Students, nothing is free and every option on Azure costs money. If it is a student subscription, the free Postgres offer exists and he's selecting the wrong tier.

That's one screen: portal → **Free Services**. If PostgreSQL appears there, it's free and correctly configured; if the page is empty, the subscription is ineligible and DigitalOcean's \$200 credit becomes the sensible answer.

Worth asking him directly: what does the subscription say under Subscriptions → Overview? "Azure for Students" or "Pay-As-You-Go"? Everything else follows from that.

my work has a paid Supabase, and theres one instance left in the project, but do then have to give me whole group access to the org?

Technically that would work — a paid Supabase org gives you 8 GB and pooled compute, and you'd just create a project in it. But I'd advise against it, and the access question is only part of why.

On access specifically: Supabase invites are org-level or project-level. You'd want project-level for the five others, which limits them to that one project rather than everything your employer has in there. So the answer to your literal question is no, you don't have to grant org access. But the person doing the inviting has to be someone with permission to invite, and that's likely you or someone at your work.

The bigger problems:

Your employer's Supabase org is their infrastructure and probably their billing. Putting a university group project in it, with five non-employees holding credentials, is the kind of thing that needs someone at work to say yes explicitly — not "there's a spare instance." If it's a client environment or has other people's data in adjacent projects, that's a stronger objection again.

It also creates a single point of failure that isn't the group's. If you leave, or your work revokes access, or someone reorganises the org, your project's database vanishes mid-semester. ADR-003 already flags that you have no backups.

And you'd be documenting, in an assessed decision record, that the platform runs on a company's paid account obtained through a member's employment. That's awkward to write honestly and worse to leave out.

What I'd do: treat it as a genuine last resort, and only with explicit written permission from whoever owns that org. The Azure student offer, if it applies, is free and belongs to the group. DigitalOcean's \$200 credit covers you to submission. Both are cleaner.

Is Dean's subscription actually Azure for Students? That still hasn't been established, and it decides everything.

i think we will use the project on my work's thing, how do we do?

Before any steps: **have you actually asked permission?** Not "there's a spare instance" but someone with authority saying yes to five external people holding credentials in their org for three months. If that conversation hasn't happened, it's the first step, and it might change the answer.

Assuming it has, or you're doing it now:

1. Create the project. Supabase dashboard → your work's org → New project. Name it something obviously not-work, like `sport-analytics-tool-wits`. **Region: choose the closest available** — if `af-south-1` or anything in Africa is offered, take it; that alone would cut your 185 ms. Set a strong database password and put it somewhere the group can reach that isn't WhatsApp.

2. Disable the Data API at creation. This is non-negotiable — Brendan's ruling permits hosted Postgres over a connection string with PostgREST never used, and ADR-003 says disabling it at creation is a stronger guarantee than agreeing not to call it. Same posture as the current project.

3. Invite the five others at project level, not org level. Settings → Team → Invite, and scope to the project.

4. Get the pooler connection string. Session pooler, port 5432 — the direct host may not resolve, same as before.

5. Migrate and seed. From your machine, change `DATABASE_URL`, then `db:migrate`, `db:seed`, `db:validate`. Twenty minutes and no data to move.

6. Check the TLS certificate. A new project may use a different CA than the one committed at `apps/backend/certs/supabase-ca.crt`. If connections fail on certificate verification, download the new one — do not disable verification to make it work.

7. Write ADR-005 recording the move, the permission you obtained, the region, and the risk that the org isn't the group's.

Step 7 matters more than it looks. This arrangement needs to be documented honestly, including the dependency on your employment.

yes i asked my boss and he said its five

Good — that's the part that needed a person, not a process.

Two things worth doing while it's fresh: get his yes in writing (a Slack or email line is enough) and store it as evidence, the way you did with Brendan's Supabase ruling at `evidence/decisions/2026-08-05-lecturer-ruling-supabase.md`. It costs nothing now and matters if anyone asks later. Worth also pinning down whether it's fine for the whole semester or something to revisit.

Step 1 — create the project.

In the Supabase dashboard, under your work's org: **New project.**

- Name: `sport-analytics-tool-wits`
- Database password: strong, generated, saved somewhere the group can reach that isn't WhatsApp
- **Region: pick the closest to South Africa available.** Check the dropdown for `af-south-1` (Cape Town). If it's there, take it — that alone could drop your 185 ms round trip to under 20 ms. If not, note which regions are offered and tell me.
- **Data API: disable it at creation** if the option is presented. This is the compliance requirement from Brendan's ruling.

Tell me the region you got and whether the Data API toggle appeared at creation. Then we do the connection string and migrations.

im using eu-west and data api was toggled off

Data API off at creation is the right call — that's the compliance position preserved.

One thing on the region: eu-west is Ireland, which is roughly the same distance as Frankfurt. Your 185 ms latency won't improve. Was `af-south-1` (Cape Town) in the dropdown and you had a reason not to take it, or wasn't it offered? Worth asking because the project can't be moved later without recreating it, and this is the cheapest moment to fix it.

If it wasn't offered, fine — nothing lost relative to today, and the storage problem is solved either way.

Step 2 — get the connection string.

Dashboard → **Connect** (top of the project page). You want the **Session pooler** string on port 5432, not the direct connection — the direct host doesn't publish an A record and fails name resolution, which is documented in ADR-003.

It'll look like:

```
postgresql://postgres.<project-ref>:[YOUR-PASSWORD]@aws-0-eu-west-1.pooler.supabase.com:5432/postgres
```

Don't paste it here with the password in it. Just confirm you have it and that it's the session pooler on 5432.

Then update `apps/backend/.env`:

```
DATABASE_URL=<the pooler string with the real password>
```

And tell me — I'll give you the migration and TLS check next.

done

Step 3 — test the connection before migrating. The TLS certificate is the thing most likely to break, since a new project may use a different CA than the one committed.

```
powershell
```

```
npm run db:latency --workspace=@sport-analytics/backend
```

That connects and runs twenty queries, so it tests TLS and gives you the new region's latency in one go.

Paste the output.

If it fails on certificate verification, don't disable verification — we'll get the right CA. If it connects, note the median: it should be similar to before given eu-west, but worth knowing the actual number for ADR-005.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:latency --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:latency
> tsx --env-file=.env scripts/latency.ts
getaddrinfo ENOTFOUND db.xgfc1pxltjsmmwcnhkf.supabase.co
npm error Lifecycle script db:latency failed with error:
npm error code 1
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
scripts/latency.ts
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

That's the direct connection string, not the session pooler — `db.<ref>.supabase.co` is the direct host, and it publishes no A record, which is exactly the failure ADR-003 documents from your #13 work.

Go back to **Connect** in the dashboard and pick the **Session pooler** tab. Its host looks like:

```
aws-0-eu-west-1.pooler.supabase.com
```

and the username has the project ref embedded — `postgres.xgfc1pxltjsmmwcnhkf` rather than plain `postgres`. Port 5432.

Update `DATABASE_URL` in `apps/backend/.env` with that, then rerun:

```
powershell

npm run db:latency --workspace=@sport-analytics/backend
```

Incidentally, this is a good sign for your documentation — the exact failure ADR-003 predicted, reproduced on a new project. Worth a line in ADR-005 that the finding generalises rather than being specific to the old instance.

```
postgresql://postgres:[YOUR-  
PASSWORD]@db.xgfc1pxltjsmmwicznhkf.supabase.co:5432/postgre  
s  
this one?
```

No — that's the direct connection again, the one that just failed. Same `db.<ref>.supabase.co` host.

In the **Connect** dialog there are usually tabs or a dropdown: **Direct connection**, **Transaction pooler**, **Session pooler**. You want **Session pooler**.

Tell them apart by the host and username:

- Direct: `postgres @ db.<ref>.supabase.co` ← what you've pasted twice
- Session pooler: `postgres.<ref> @ aws-0-eu-west-1.pooler.supabase.com:5432`
- Transaction pooler: same host but port **6543** — don't use this one, it doesn't support prepared statements properly and the migration tooling will misbehave

The giveaway is the username: if it's plain `postgres`, it's direct. If it's `postgres.xgfc1pxltjsmmwicznhkf`, it's a pooler.

If you can't find a session pooler option at all, say so — newer projects sometimes present this differently, and there's a chance IPv4 direct connections now work on this project, in which case the failure was something else.

```
postgresql://postgres.xgfc1pxltjsmmwicznhkf:[YOUR-PASSWORD]@aws-0-eu-  
west-2.pooler.supabase.com:5432/postgres  
this?
```

That's the one — pooler host, project ref in the username, port 5432.

Put it in `apps/backend/.env` as `DATABASE_URL` with the real password substituted for `[YOUR-PASSWORD]`, then:

```
powershell  
  
npm run db:latency --workspace=@sport-analytics/backend
```

Paste the output.

If the password contains special characters (`@`, `#`, `/`, `:`), they need URL-encoding or the string parses wrong — tell me if it fails oddly and we'll check that.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:latency --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:latency
> tsx --env-file=.env scripts/latency.ts
connect: 1171 ms
queries: 20
  min   : 172 ms
  median: 173 ms
  max   : 176 ms
  mean  : 173.6 ms
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Connected, and the TLS certificate worked without changes — the committed CA covers this project too.

173 ms median, versus 181 ms on the old one. Essentially unchanged, as expected for eu-west-2 (London) versus Frankfurt. Storage problem solved, latency problem not.

Step 4 — migrate:

```
powershell
npm run db:migrate --workspace=@sport-analytics/backend
```

Step 5 — seed:

```
powershell
npm run db:seed --workspace=@sport-analytics/backend
```

Step 6 — validate:

```
powershell
npm run db:validate --workspace=@sport-analytics/backend
```

Run them in order and paste the results — the migration output is long, so just the last few lines is fine. Validation passing all 22 assertions against a fresh database is the proof the move is complete, and it's good evidence for ADR-005.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm
run db:migrate --workspa
ce=@sport-analytics/bac
kend > @sport-analytics/
backend@0.1.0 db:migr...
```

PASTED

Clean move. Migrations applied from empty, seed loaded 955 deliveries, and all 22 assertions pass against the published scorecard on a database that didn't exist an hour ago.

Worth noting what that actually demonstrates: the schema is portable, the migrations are complete, and the seed is genuinely reproducible. Most groups couldn't move providers mid-project in twenty minutes.

Also — 3 wickets for Sandeep Sharma, matching the published player-of-the-match line of 3/21, and 2 dismissals not credited to any bowler. My earlier guess of 17 credited was wrong; it's 17 credited across nine bowlers, 2 uncredited, 19 total. Good thing the script computes it rather than trusting me.

Now the parts that make this real for everyone else:

1. The four env vars. Everyone needs the new `DATABASE_URL`. Share it however your group shares secrets — not the group chat.

2. Azure App Service needs it too, or the deployed backend points at the old database.

3. ADR-005. This is a decision record: why you moved, what the constraint was, what you rejected (Oracle, DigitalOcean, Azure Flexible Server, paying), the permission you obtained from your employer, and the risks — that the org isn't the group's, that access depends on your employment, and that latency is unchanged at 173 ms.

4. Update ADR-003 consequence 4 to record that the storage constraint is resolved and by what.

Want me to draft ADR-005?

is that all it took? its done? wow that wa squick

The technical move was quick because you'd already done the hard part. The schema lives entirely in committed migrations, the seed is four files in the repo, and nothing in the application uses a provider-specific interface — that's exactly what ADR-003 said would make changing host cheap, and it turned out to be true. It's the payoff for decisions made a week ago.

But **it's not done**, and the gap between "works on Ben's machine" and "done" is where this could go wrong:

- Five people still have `DATABASE_URL` pointing at the old database. Right now you're the only one on the new one.
- Azure App Service still points at the old one — so when the deployment eventually works, it'll hit the wrong database.
- The old Supabase project still exists with data in it, and nobody should be writing to it. It needs a decision: keep for a week, then delete.
- ADR-003 says a tested dump and restore procedure is required. Still doesn't exist, and now the database lives in your employer's org, which raises the stakes.

- No decision record exists for any of this yet.

The last one matters most for your marks. Sprint 1 is ~80% process, and an undocumented provider migration reads as chaos rather than competence — whereas ADR-005 explaining the constraint, the options, the permission and the risks reads as exactly the opposite.

Say the word and I'll draft it. Then the group message with the new connection details.

okay, lets go step by step, tell me what to do

Step 1 — create the issue, since a provider migration needs one under your methodology.

Title: Move the database to a Supabase project with sufficient storage

Type: Task. **Body:**

markdown

Description

Move the project database from the free Supabase project to a project in an organisation with a paid plan, resolving the storage constraint recorded in ADR-003 consequence 4.

Motivation

ADR-003 consequence 4 records that the agreed competition scope is likely to exceed the free plan's 500 MB limit, and requires the team to decide between paying for additional storage, reducing scope, or moving provider. The decision has had no owner since 5 August 2026 and has blocked bulk ingestion, the storage benchmark required by the same record, and the automated reference-fixture test.

Acceptance Criteria

- [] A new project exists with the Data API disabled at creation
- [] Migrations apply from empty against the new database
- [] The development seed loads and is repeatable
- [] Reference fixture 729307 validates against the published scorecard
- [] Every team member's `DATABASE\_URL` is updated
- [] The Azure App Service configuration is updated
- [] ADR-005 records the decision, the alternatives rejected, and the risks
- [] ADR-003 consequence 4 is updated to record the resolution
- [] A date is agreed for deleting the old project

Dependencies

Written permission from the owner of the hosting organisation.

Evidence

Migration, seed and validation output against the new database. ADR-005.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Cl

Labels: area: database, area: infrastructure, priority: high. Milestone Sprint 1.
Assign yourself. Board: In Progress.

Tell me the issue number and I'll give you Step 2.

Database round-trip latency is 185 ms and constrains API response times #105
Open
opened 2 days ago by BenSwartz · 0 comments
BenSwartz commented 2 days ago
Member

Description

Every database query from a developer machine in Johannesburg to the hosted Supabase instance costs approximately 185 ms of round-trip latency. This is a property of the distance between the client and the database region, not of the queries.

Measured on 10 August 2026 with `apps/backend/scripts/latency.ts`, twenty sequential `SELECT 1` statements over an established connection:

Measure Value

Connection setup 1233 ms

Minimum 173 ms

Median 181 ms

Mean 185.8 ms

Maximum 248 ms

Motivation

The intermediate tier requires the platform to meet a stated response time under load. With a 185 ms floor per query, an endpoint issuing three sequential queries starts at 555 ms before any query work is done. No amount of indexing or schema design reduces this; the only levers are issuing fewer queries per request, serving repeated reads from cache, and the distance between the application and the database.

The same floor explains the ingestion rate. Loading one fixture takes approximately 68 seconds, which is roughly 290 round trips at 185 ms. That is a separate issue, but it has the same cause.

ADR-003 already records latency as a consequence of the hosted database decision, on the basis that the application will be deployed to Azure South Africa North while the database is in eu-central-1. This issue supplies the measurement that consequence lacked.

Acceptance Criteria

The measurement is recorded in ADR-003 alongside the consequence it quantifies

Endpoints are written to issue as few sequential queries per request as the work allows, rather than one query per entity

A target response time is stated and tested against, taking the latency floor into account

The team decides whether the latency, together with the storage limit, justifies revisiting the database host

Dependencies

Relates to the storage decision in ADR-003 consequence 4. Both point at the same

question: whether the current hosted instance is the right place for this platform. Neither has an owner.

Evidence

`apps/backend/scripts/latency.ts` and the figures above. The script is committed so the measurement can be repeated from any member's machine, which will also show whether the figure varies by location.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Claude Opus 5].

i made this?

Show more

Yes, that's #105 — the latency issue you created on 10 August. Still open, and its fourth acceptance criterion is directly relevant: *"The team decides whether the latency, together with*

the storage limit, justifies revisiting the database host."

You've now half-answered it. The storage limit is resolved; latency is unchanged at 173 ms, because eu-west-2 is London and eu-central-1 was Frankfurt.

Worth a comment on #105 recording that:

markdown

Partly addressed. The database has moved to a project in an organisation with a paid plan, resolving the storage constraint in ADR-003 consequence 4. The new project is in `eu-west-2`, so the latency is materially unchanged: 173 ms median over twenty samples, against 181 ms on the previous project.

The storage half of the question in the fourth criterion is therefore settled and the latency half is not. Reducing it would require a database in or near South Africa, which no free or already-available option currently offers.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

But that's a side quest. **The step you're on is creating the migration issue** — the one I drafted in my last message. Create it, tell me the number, and we carry on.

Move the database to a Supabase project with sufficient storage #121

Open

opened now by [BenSwartz](https://sdp.ms.wits.ac.za/BenSwartz) · 0 comments

[BenSwartz](https://sdp.ms.wits.ac.za/BenSwartz) commented [now]

(https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/issues/121#issue-303)

Member

Description

Move the project database from the free Supabase project to a project in an organisation with a paid plan, resolving the storage constraint recorded in ADR-003 consequence 4.

Motivation

ADR-003 consequence 4 records that the agreed competition scope is likely to exceed the free plan's 500 MB limit, and requires the team to decide between paying for additional storage, reducing scope, or moving provider. The decision has had no owner since 5 August 2026 and has blocked bulk ingestion, the storage benchmark required by the same record, and the automated reference-fixture test.

Acceptance Criteria

- A new project exists with the Data API disabled at creation
- Migrations apply from empty against the new database
- The development seed loads and is repeatable
- Reference fixture 729307 validates against the published scorecard
- Every team member's `DATABASE_URL` is updated
- The Azure App Service configuration is updated
- ADR-005 records the decision, the alternatives rejected, and the risks
- ADR-003 consequence 4 is updated to record the resolution

- A date is agreed for deleting the old project

Dependencies

Written permission from the owner of the hosting organisation.

Evidence

Migration, seed and validation output against the new database. ADR-005.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Claude Opus 5].

Show more

#121. Tick the first four criteria now — they're already done and verified.

Step 2 — create the branch:

powershell

```
git switch main
git pull origin main
git switch -c docs/121-move-database-host
```

Paste the output — the pull may bring in changes and I want to know `main` is current before we write documentation that references it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main
>> git pull origin main
>> git switch -c docs/121-move-database-host
Switched to branch 'main'
Your branch is behind 'origin/main' by 14 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
remote: Failed to authenticate user
fatal: Authentication failed for 'https://sdp.ms.wits.ac.za/git-push-pray/Sport-
Analytics-Tool.git/'
Switched to a new branch 'docs/121-move-database-host'
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

The pull failed on the intermittent auth error, so your branch is 14 commits behind. Fix that before writing anything:

powershell

```
git switch main
git pull origin main
git branch -D docs/121-move-database-host
git switch -c docs/121-move-database-host
```

Paste the output. If auth fails again, retry — it's always cleared on retry so far.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main >> git pull origin main >> git branch -D docs/121-move-database-host >> git switch -c docs/121-move-database-host >> git switch -c docs/121-move-database-host
```

PASTED

Up to date and on the branch. Shayna's localStorage fix landed too, so **main** should now pass **check** for you.

Step 3 — check the ADR numbering and how existing ones are structured, so ADR-005 matches:

```
powershell
```

```
ls evidence\decisions
```

```
powershell
```

```
Select-String -Path evidence\decisions\ADR-004*.md -Pattern "^#|^##" | Select-Object -
```

Paste both. I want to confirm 005 is free and see the section headings the others use before drafting.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> ls evidence\decisions
Directory: C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\decisions
Mode                LastWriteTime         Length Name
----                -
-a----          06/08/2026 11:24           975 2026-08-05-lecturer-ruling-supabase.md
-a----          05/08/2026 17:13           271 ADR-000-template.md
-a----          05/08/2026 17:13          5680 ADR-001-initial-repository-setup.md
-a----          06/08/2026 16:53          8005 ADR-002-firebase-authentication-
foundation.md
-a----          10/08/2026 15:03          7662 ADR-003-database-host-connection-and-
migrations.md
-a----          10/08/2026 15:03          8740 ADR-004-supabase-authentication-
foundation.md
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

005 is free. Show me the template so ADR-005 matches the house structure:

```
powershell
```

```
Get-Content evidence\decisions\ADR-000-template.md
```

Paste it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
evidence\decisions\ADR-000-template.md
# ADR-000: Decision title
- **Status:** Proposed
- **Date:** YYYY-MM-DD
- **Participants:**
- **Related issues:**
## Context
## Decision
## Alternatives considered
## Advantages
## Disadvantages
## Consequences
## Verification and review date
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Step 4 — create ADR-005. Run this from the repository root:

```
powershell
```

@'

ADR-005: Move the database to a Supabase project with sufficient storage

- ****Status:**** Accepted
- ****Date:**** 2026-08-11
- ****Participants:**** Ben Swartz
- ****Related issues:**** #121, #105, #27, #13

Context

ADR-003 consequence 4 records that the agreed competition scope covers eleven franchise competitions plus men's and women's T20 internationals, that the T20 international corpus alone is 1,266,835 deliveries at an estimated 500 MB with indexes, and that the full scope is likely to exceed the free plan's limit. It requires the team to decide between paying for additional storage, reducing scope, or moving provider, and states that the decision must be recorded before ingestion begins.

The decision had no owner and no date from 5 August 2026. In the intervening period it blocked bulk ingestion, the measured storage benchmark the same record requires, and the automated reference-fixture test, which needs a separate database whose name contains "test" and which could not be created against a 500 MB limit already carrying the development data.

A partial measurement taken on 7 August over four fixtures gave approximately 128 bytes per delivery of table heap. It could not confirm the 414 bytes per delivery estimate, because every other table sat at the minimum page allocation and index sizes were at their floor. The estimate therefore remains unverified, which is itself a reason to prefer an option that removes the constraint over one that fits it.

Decision

The database moves to a new Supabase project created in an organisation holding a paid plan, made available for this project with the permission of its owner.

The new project is in `eu-west-2`. The Data API was disabled at creation, so the generated endpoints do not exist, consistent with the client ruling recorded at `evidence/decisions/2026-08-05-lecturer-ruling-supabase.md` and with ADR-003 consequence 1. Connection is through the session pooler on port 5432, for the reason established in ADR-003: the direct connection host publishes no A record and fails name resolution.

Alternatives considered

****Azure Database for PostgreSQL Flexible Server.**** An Azure free account provides this service free for twelve months with 750 hours of Burstable B1MS compute and 32 GB of storage. Placing it in South Africa North would also have removed the

latency floor recorded in #105, since the application is deployed to that region. This was the strongest option on the merits. It was not taken because the team's Azure subscription showed charges rather than the free allowance, and eligibility for the free offer was not established.

****Supabase Pro.**** \$25 per month for 8 GB, approximately \$50 to final submission. Rejected because a free option was available and it required a recurring charge owned by an individual member.

****Oracle Database Free.**** Proposed and rejected. The offer is a download rather than a hosted service, and the schema depends throughout on PostgreSQL features with no Oracle equivalent: enumerated types, `text[]`, partial unique indexes, `UNIQUE NULLS NOT DISTINCT`, `num\_nonnulls`, `ON CONFLICT DO NOTHING` on which idempotent ingestion depends, and `COUNT(\*) FILTER` on which most derivation depends. Migration tooling and the driver would also have been replaced. The cost would have been a rewrite of every layer below the API, including work owned by other members.

****DigitalOcean Managed PostgreSQL.**** Real PostgreSQL, so no rewrite, and \$200 of credit for 60 days covers the project to approximately mid-October. Rejected because it charges \$15 per month thereafter, adds a further provider, and does not reduce latency.

****Reducing scope.**** The project brief requires the platform to be exercised at hundreds of fixtures with hundreds of events each. At the measured heap figure the previous 500 MB limit already accommodated on the order of a thousand matches, so the constraint may have arisen from a scope decision larger than what is assessed. This option remains open on its merits and is not foreclosed by this record.

Advantages

- Removes the storage constraint that has blocked three separate pieces of work since 5 August.
- Costs nothing.
- No change to any application code. The schema is defined entirely in committed migrations, the development seed is four match files committed to the repository, and no provider-specific interface is used anywhere.
- Preserves the compliance position exactly: the Data API is disabled at creation rather than merely unused.
- The previous project remains intact during the transition, so the move is reversible by changing one environment variable back.

Disadvantages

- The hosting organisation belongs to a member's employer rather than to the team. Access depends on that member's continued employment and on permission that could be withdrawn.
- Latency is unchanged. `eu-west-2` is comparable in distance to the previous

- `eu-central-1`: 173 ms median round trip against 181 ms, both measured over twenty samples. The issue recorded in #105 is not addressed by this decision.
- The project now depends on two Supabase organisations, since authentication remains on the previous project under ADR-004. This coupling is accepted deliberately rather than by oversight and should be revisited.
 - No backups exist. ADR-003 consequence 5 already requires a tested dump and restore procedure and none has been written. Hosting in another party's organisation raises rather than lowers the importance of this.

Consequences

1. Every member must update `DATABASE\_URL`. Until they do, members are working against two different databases, and counts will disagree.
2. The Azure App Service configuration must be updated, or the deployed backend will read the previous database once deployment is working.
3. The previous project must not be written to after the transition, and a date for deleting it must be agreed. Until it is deleted it remains a source of confusion and a second place where project data exists.
4. ADR-003 consequence 4 is superseded by this record on the storage question. The latency consequence in the same record stands unchanged.
5. The unverified 414 bytes per delivery estimate is no longer load-bearing for any decision, but the measured benchmark ADR-003 requires is still outstanding, and should now be taken because ingestion is no longer blocked by storage.
6. The permission obtained is informal. It should be confirmed in writing and stored alongside the client ruling in `evidence/decisions/`.

Verification and review date

Verified on 11 August 2026 against the new database: both migrations applied from empty; the development seed loaded 955 deliveries across four fixtures; reference fixture 729307 passed all twenty-two assertions against the published scorecard, including the complete ten-row fall of wickets and the exclusion of run outs from bowler credit; and the committed certificate authority validated the connection without change.

To be reviewed at the Sprint 2 close-out, or immediately if permission to use the hosting organisation is withdrawn.

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of Claude-Web[Claude Opus 5].

'@ | Set-Content -Path evidence\decisions\ADR-005-database-host-migration.md -Encoding

Check it wrote cleanly:

powershell

```
Get-Content evidence\decisions\ADR-005-database-host-migration.md -TotalCount 12
```

Two things I've deliberately included that you might want to soften — say so and I'll change them. The disadvantage about depending on your employment is stated plainly, and consequence 6 says the permission is informal and needs to be in writing. Both are true and both read better as acknowledged risks than as omissions a marker spots.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
evidence\decisions\ADR-005-database-host-migration.md -TotalCount 12
# ADR-005: Move the database to a Supabase project with sufficient storage
- **Status:** Accepted
- **Date:** 2026-08-11
- **Participants:** Ben Swartz
- **Related issues:** #121, #105, #27, #13
## Context
ADR-003 consequence 4 records that the agreed competition scope covers eleven
franchise competitions plus men"s and women"s T20 internationals, that the T20
international corpus alone is 1,266,835 deliveries at an estimated 500 MB with
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

The doubled apostrophes came through literally — `men''s` instead of `men's`. PowerShell's escaping applies inside `@"...@"` but not `@'...'@`, and I got that backwards.

Fix all of them at once:

```
powershell
(Get-Content evidence\decisions\ADR-005-database-host-migration.md -Raw) -replace "''"
```

Then verify:

```
powershell
Select-String -Path evidence\decisions\ADR-005-database-host-migration.md -Pattern "''
```

That should return nothing. Paste the result.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> (Get-Content
evidence\decisions\ADR-005-database-host-migration.md -Raw) -replace "''", "'" |
Set-Content evidence\decisions\ADR-005-database-host-migration.md -Encoding
utf8
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
evidence\decisions\ADR-005-database-host-migration.md -Pattern "'"
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Clean.

Step 5 — update ADR-003 consequence 4. It currently says the decision must be recorded before ingestion begins and that it remains outstanding. That's now false, and a decision record contradicting a newer one is worse than either alone.

Open `evidence/decisions/ADR-003-database-host-connection-and-migrations.md`, find the end of consequence 4 — the paragraph beginning "Fourteen fixtures have been loaded for schema validation" — and append after it, at the same indentation:

markdown

```
**Superseded on 11 August 2026 by ADR-005.** The database moved to a project in an organisation holding a paid plan, resolving the storage constraint. The measured benchmark this record requires remains outstanding, but is no longer blocked. The latency consequence recorded above stands unchanged: the new project is in `eu-west-2` and the median round trip is 173 ms.
```

Then add ADR-005 to the docs navigation if ADRs are published — check first:

powershell

```
Select-String -Path mkdocs.yml -Pattern "ADR|decision"
```

Paste that, and confirm the ADR-003 edit is saved.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path mkdocs.yml -
Pattern "ADR|decision"
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

ADRs aren't published to the docs site — they live in `evidence/` only, so no nav change needed.

Worth noting as an observation rather than acting on tonight: your architecture documentation references ADR-002 through ADR-004 as reasoning, but a reader of the public documentation site can't see any of them. Whether decision records should be evidence-only or published is a group call, not yours to change unilaterally.

Step 6 — commit. Confirm the ADR-003 edit saved first:

powershell

```
git status -s
```

Then:

powershell

```
git add evidence/decisions/ADR-005-database-host-migration.md evidence/decisions/ADR-C
```

powershell

```
git commit -m "docs(adr): record the database host migration" -m "ADR-005 records the  
Verified against the new database: both migrations applied from empty, the seed loaded  
ADR-003 consequence 4 is marked superseded on the storage question. Its latency conseq  
Refs #121  
Assisted-by: Claude-Web[Claude Opus 5]"
```

powershell

```
git push -u origin docs/121-move-database-host
```

Paste the output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git status -s  
M evidence/decisions/ADR-003-database-host-connection-and-migrations.md  
?? evidence/ai/transcripts/ben-swartz/AnalogueVentures_email_signature.png  
?? evidence/decisions/ADR-005-database-host-migration.md  
?? site/adr/  
?? site/ai_policy.pdf  
?? site/architecture/system-architecture/  
?? site/data/  
?? site/database/erd/  
?? site/deployment/azure-backend/  
?? site/deployment/azure-fronted/  
?? site/deployment/cloudflare_pages/  
?? site/design/  
?? site/environment/  
?? site/planning/  
?? site/security/auth-provider-comparison/  
?? site/security/authentication/  
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git add evidence/decisions/ADR-005-  
database-host-migration.md evidence/decisions/ADR-003-database-host-  
connection-and-migrations.md  
warning: in the working copy of 'evidence/decisions/ADR-005-database-host-  
migration.md', CRLF will be replaced by LF the next time Git touches it  
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git commit -m "docs(adr): record the  
database host migration" -m "ADR-005 records the move to a Supabase project in an  
organisation holding a paid plan, the alternatives rejected, and the risks. The storage  
constraint in ADR-003 consequence 4 had no owner from 5 August and blocked bulk  
ingestion, the storage benchmark and the automated reference-fixture test.
```

```

>>
>> Verified against the new database: both migrations applied from empty, the seed
loaded 955 deliveries across four fixtures, and reference fixture 729307 passed all
twenty-two assertions against the published scorecard.
>>
>> ADR-003 consequence 4 is marked superseded on the storage question. Its
latency consequence stands: the new project is in eu-west-2 and the median round
trip is 173 ms against 181 ms previously.
>>
>> Refs #121
>> Assisted-by: Claude-Web[Claude Opus 5]"
[docs/121-move-database-host 1a883b4] docs(adr): record the database host
migration
2 files changed, 146 insertions(+)
create mode 100644 evidence/decisions/ADR-005-database-host-migration.md
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git push -u origin docs/121-move-
database-host
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 4.43 KiB | 2.21 MiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'docs/121-move-database-host':
remote: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/new/docs/121-move-database-host
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
* [new branch] docs/121-move-database-host -> docs/121-move-database-host
branch 'docs/121-move-database-host' set up to track 'origin/docs/121-move-
database-host'.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

```

Pushed — but **only ADR-005 was committed**. The commit says `2 files changed, 146 insertions` with just the one file created, and no modification to ADR-003 listed. Your ADR-003 edit was staged before it was saved, or wasn't made.

Check:

```
powershell
```

```
Select-String -Path evidence\decisions\ADR-003-database-host-connection-and-migrations
```

If that returns nothing, make the edit now and commit it as a second commit on the same branch.

Also — there's a stray file in your transcripts folder: [evidence/ai/transcripts/ben-swartz/AnalogueVentures_email_signature.png](#). Not yours to commit, presumably landed there by accident. Worth deleting before it gets swept into something.

Step 7 — open the PR. Title: [#121 Record the database host migration](#)

markdown

Purpose

ADR-003 consequence 4 required the team to decide between paying for storage, reducing scope, or moving provider, and stated the decision must be recorded before ingestion begins. It had no owner from 5 August and blocked bulk ingestion, the storage benchmark the same record requires, and the automated reference-fixture test.

Changes

- `ADR-005` records the move to a Supabase project in an organisation holding a paid plan, the four alternatives rejected and why, and the risks.
- ADR-003 consequence 4 marked superseded on the storage question.

Testing

Verified against the new database on 11 August: both migrations applied from empty; the seed loaded 955 deliveries across four fixtures; reference fixture 729307 passed all twenty-two assertions against the published scorecard, including the full fall of wickets and the exclusion of run outs from bowler credit; the committed certificate authority validated the connection unchanged.

Known Limitations

Latency is not addressed. The new project is in `eu-west-2`, median round trip 173 ms against 181 ms previously. #105 stands.

The hosting organisation belongs to a member's employer. Access depends on that member's employment and on permission that could be withdrawn. The permission is currently informal and should be confirmed in writing.

Authentication remains on the previous project under ADR-004, so the project now depends on two organisations.

Everyone still needs to update `DATABASE\_URL`, and the Azure App Service configuration is unchanged.

Related Issue

Refs #121

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
evidence\decisions\ADR-003-database-host-connection-and-migrations.md -
Pattern "Superseded on 11 August"
evidence\decisions\ADR-003-database-host-connection-and-migrations.md:121:
**Superseded on 11 August 2026 by ADR-005.**
The database moved to a project
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

It's there — the edit did commit. I misread `2 files changed, 146 insertions` as only counting the new file; the ADR-003 change was in there too.

So the branch is complete. Open the PR with the description from my last message, then request a review.

Two things after that:

Delete the stray PNG — `evidence/ai/transcripts/ben-swartz/AnalogueVentures_email_signature.png` is untracked and shouldn't be in the transcripts folder.

Tell the group about `DATABASE_URL`. Right now you're the only person on the new database, and everyone else is silently working against the old one. Their counts will disagree with yours and neither side will know why. That's the most urgent remaining item — more than the PR.

Want the message?

yes, and then is the issue closed?

A Full context B Shorter

Heads up — I've moved our database. We were on the free Supabase plan with a 500MB limit and our corpus needs way more than that, which has been blocking bulk ingestion since the 5th. My work has a paid Supabase org and my boss okayed us using a project in it, so that's where it is now.

****Everyone needs to update DATABASE_URL in apps/backend/.env.**** I'll send the connection string separately, not in here. Until you do, you're pointing at the old database and your counts won't match mine.

Already done and verified on the new one: migrations applied from empty, seed loaded (4 fixtures, 955 deliveries), and match 729307 passes all 22 assertions against the published scorecard. So it's working, not just connected.

Nothing in the code changed — same Postgres, same schema, same certificate. Literally just the one env var.

ADR-005 is up in a PR (#121) with the reasoning and the alternatives we didn't take. Worth a skim if you're curious why not Oracle or Azure.

Two things that aren't fixed by this:

- Latency is the same, 173ms vs 181ms — new project's in London, old one was Frankfurt. #105 still stands.
- @Dean the Azure App Service still points at the old DB, needs updating when you get a chance



On closing #121 — **no, not yet**. Four of its nine criteria are outstanding, and they're the ones that make the move real rather than local to your machine:

- Every member's `DATABASE_URL` updated — nobody's done it
- Azure App Service configuration updated — Dean's
- ADR-005 recorded — done, but the PR isn't merged
- A date agreed for deleting the old project — no date

Tick the four that are genuinely done (project created with Data API disabled, migrations from empty, seed loads and repeats, 729307 validates) plus ADR-003 updated, and leave it In Progress.

The old project is the one I'd not let drift. Two databases with the same schema and overlapping data is exactly how someone spends an afternoon debugging a count that was right all along.

just checked, they all have the new URL, what next?

Then tick that criterion. Two left before #121 closes:

1. Azure App Service — Dean's, and it's a one-line config change. Worth chasing now while he's around rather than discovering it during the next deployment attempt.

2. A date for deleting the old project — needs the group, not you. Propose one and let people object: a week from now, say 18 August, gives everyone time to notice anything they've left behind.

Both are asks rather than work, so they can go in one message. Then the PR needs a review before it merges, which is the third outstanding thing.

After that, what's actually left on your plate is short:

- #63 and #43 parked on decisions that aren't yours
- #105 half-answered — storage settled, latency not
- The storage benchmark ADR-003 has always wanted, now unblocked

That last one is the interesting one. With storage no longer a constraint, you could load a few hundred fixtures and finally get the measured figure the ADR has been asking for since 5 August — except ingestion at 68 seconds a match makes that a five-hour job. Which brings you back to batching the ingester, the thing we deferred.

Want to chase Dean and propose the deletion date first, or start on the ingester?

do we not have to keep old project for Oauth?

Yes — good catch, and it's a correction to what I've been saying.

Authentication runs on the old project under ADR-004: `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY` on the backend and the `VITE_` equivalents on the frontend all point there. Delete it and sign-in breaks for everyone.

ADR-005 does note the two-organisation coupling as a disadvantage, but I told you to propose a deletion date immediately after — which would have broken auth. Don't send that.

So the old project stays, and the position is: **auth on the old project, data on the new one.**

That's a real dependency, not a transitional state, and it has consequences worth naming:

- `app_user.auth_subject` in the new database references identities that live in the old project. If the old project ever goes, those links dangle.
- The old project is still on the free plan, which pauses after a week of inactivity. Auth traffic probably keeps it alive, but that's an assumption worth testing rather than trusting.
- Two projects, two sets of credentials, two things that can be deleted by accident.

The cleaner end state is moving auth to the new project too — same organisation, one place, and it removes the dangling-reference risk. That's Gabriel's territory since he owns #14 and ADR-004, and it needs a migration of existing users or a decision that there aren't any worth keeping.

Two things to do: amend the deletion-date criterion on #121 to say the old project must be retained until authentication moves, and raise the auth move with Gabriel.

Want me to draft the #121 comment?

Step 1 — comment on #121 correcting the deletion criterion:

markdown

Correction to the last acceptance criterion. The previous project cannot be deleted: authentication runs on it under ADR-004, and both the backend `SUPABASE\_URL` and `SUPABASE\_PUBLISHABLE\_KEY` and their `VITE\_` equivalents on the frontend point there. Deleting it would break sign-in.

The position after this move is therefore authentication on the previous project and data on the new one, which ADR-005 records as a deliberate coupling rather than a transitional state. Three consequences follow:

- `app\_user.auth\_subject` in the new database references identities held in the previous project. Those references dangle if it is ever removed.
- The previous project remains on the free plan, which pauses after a week of inactivity. Whether authentication traffic is sufficient to prevent this has not been tested.
- Two projects and two sets of credentials are now load-bearing.

Replacing that criterion with: the previous project is retained until authentication is moved, and no data is written to it in the meantime.

Moving authentication to the new project would resolve all three and is raised separately.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

Then edit the issue body: change the last criterion from "A date is agreed for deleting the old project" to "The previous project is retained until authentication moves, and is not written to".

Tell me when that's done and I'll give you Step 2.

done, must i not update the publishable key and supabase url on backend .env?

No — leave them exactly as they are. They point at the old project because that's where authentication lives, and changing them to the new project would break sign-in, since no users exist there and the OAuth configuration hasn't been set up.

That's the split, and it's worth being clear about because it's confusing:

Variable	Project
<code>DATABASE_URL</code>	New
<code>SUPABASE_URL</code>	Old
<code>SUPABASE_PUBLISHABLE_KEY</code>	Old
<code>VITE_SUPABASE_URL</code>	Old
<code>VITE_SUPABASE_PUBLISHABLE_KEY</code>	Old

One variable moved. Four didn't.

Worth telling the group that explicitly, because "we moved the database" naturally reads as "update all the Supabase things," and someone will helpfully update all five and then wonder why they can't sign in.

Step 2 — chase Dean on the Azure App Service. Same point applies there: only `DATABASE_URL` changes; the Supabase auth variables stay pointing at the old project.

Then the PR still needs a review before #121 can close.

dean put it in azure

Then tick that criterion too. #121 is down to one outstanding item — ADR-005 needs its PR reviewed and merged.

Worth verifying Dean set only `DATABASE_URL` and didn't helpfully update the Supabase variables as well. The deployed backend still 503s for other reasons, so you can't test it by hitting the API — but he can tell you what he changed.

Step 3 — get the PR reviewed. It's been sitting since you pushed it. Ask whoever's around.

Once it merges, tick the ADR criterion, close #121, move the board item to Done.

Then the genuinely useful next thing: the storage benchmark ADR-003 has wanted since 5 August is finally unblocked. The obstacle is now purely ingestion speed — 68 seconds a match makes a few hundred fixtures a five-hour job. Batching the ingester would cut that to under an hour and give you a real measurement instead of an unverified estimate.

That's the piece of work I'd pick up next, unless something more urgent has come from the group.

PR was merged!

Then #121 is complete. Tick the ADR criterion, close the issue, move the board item to **Done**, and delete the branch:

powershell

```
git switch main
git pull origin main
git branch -d docs/121-move-database-host
git push origin --delete docs/121-move-database-host
```

That closes out a decision that had been blocking three separate things for six days.

Also update your register — today's work isn't recorded. That's the hosting research, ADR-005, the migration itself, and issue #121. Want the row?

Then the ingester. With storage resolved, batching it is the thing that unblocks the storage benchmark ADR-003 has always wanted — and it's a Sprint 2 prerequisite regardless.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main
>> git pull origin main
>> git branch -d docs/121-move-database-host
>> git push origin --delete docs/121-move-database-host
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
remote: Failed to authenticate user
fatal: Authentication failed for 'https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git/'
warning: deleting branch 'docs/121-move-database-host' that has been merged to
'refs/remotes/origin/docs/121-move-database-host', but not yet merged to
HEAD
Deleted branch docs/121-move-database-host (was 1a883b4).
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
- [deleted]    docs/121-move-database-host
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Branch deleted both sides, but the **git pull** failed on the usual auth flake — so your local **main** doesn't have the merge yet. Retry:

powershell

```
git pull origin main
git log --oneline -3
```

Paste that. Then the register row.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git pull origin main >> git log --oneline -3 remote: Enumerating objects: 138, done. remote: Counting object...
```

PASTED

ADR-005 is in `main`. #121 can close.

But note what else came in: **a new migration**, `20260812120000000_account-authorisation.sql`, from #44. That's Dean's account and authorisation work — and it needs applying to your new database, which was migrated before it existed.

```
powershell
```

```
npm run db:migrate --workspace=@sport-analytics/backend
```

Then confirm nothing broke:

```
powershell
```

```
npm run db:validate --workspace=@sport-analytics/backend
```

Paste both. Worth checking because this is the first migration written by someone else against your schema — if it references `app_user` or adds constraints, the validation is the quickest way to know your data is still intact.

Also worth flagging to the group: anyone who migrated before today needs to run `db:migrate` again. That'll be a recurring thing now that more than one person writes migrations.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:migrate --workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:migrate
> cross-env NODE_EXTRA_CA_CERTS=certs/supabase-ca.crt node-pg-migrate up --envPath .env --migrations-dir ../../database/migrations --ignore-pattern "README.md" --reject-unauthorized true
> Migrating files:
> - 20260812120000000_account-authorisation
### MIGRATION 20260812120000000_account-authorisation (UP) ###
-- Up Migration
-- Extend the existing provider-neutral application account with the minimum
-- server-owned state needed for approval and competition-scoped authorisation.
ALTER TABLE app_user
  ADD COLUMN submitter_approval_state text NOT NULL DEFAULT
'not_requested',
  ADD COLUMN last_authenticated_at timestamptz NOT NULL DEFAULT now(),
  ADD CONSTRAINT app_user_application_role_ck
  CHECK (application_role IN ('viewer', 'administrator')),
```

```

    ADD CONSTRAINT app_user_submitter_approval_state_ck
    CHECK (submitter_approval_state IN ('not_requested', 'pending', 'approved',
'rejected'));
CREATE TABLE submitter_competition_scope (
    app_user_id bigint NOT NULL REFERENCES app_user ON DELETE CASCADE,
    competition_id bigint NOT NULL REFERENCES competition ON DELETE
CASCADE,
    created_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (app_user_id, competition_id)
);
COMMENT ON COLUMN app_user.submitter_approval_state IS
    'Server-owned submission approval state. Authentication alone never changes this
value.';
COMMENT ON TABLE submitter_competition_scope IS
    'Server-owned competition grants for approved submitters. Client-supplied role or
scope values are never authoritative.';
CREATE INDEX submitter_competition_scope_competition_idx
    ON submitter_competition_scope (competition_id, app_user_id);
;
INSERT INTO "public"."pgmigrations" (name, run_on) VALUES
('20260812120000000_account-authorisation', NOW());
Migrations complete!
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:validate --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:validate
> tsx --env-file=.env scripts/validate-match.ts
Validating fixture 729307 against the published scorecard.
Innings 0 — Kings XI Punjab
PASS team: Kings XI Punjab
PASS runs: 132
PASS wickets: 9
PASS legal balls: 120
    overs: 20.0
PASS extras: 5
    breakdown: 1w 1nb 1b 2lb
Innings 1 — Kolkata Knight Riders
PASS team: Kolkata Knight Riders
PASS runs: 109
PASS wickets: 10
PASS legal balls: 110
    overs: 18.2
PASS extras: 10
    breakdown: 5w 0nb 0b 5lb
Fall of wickets — Kolkata Knight Riders (innings 1)
PASS 1-13 (2.4 ov) MK Pandey, lbw
PASS 2-19 (4.1 ov) G Gambhir, caught
PASS 3-19 (5.1 ov) JH Kallis, caught
PASS 4-50 (11.1 ov) CA Lynn, bowled
PASS 5-59 (12.3 ov) YK Pathan, lbw
PASS 6-62 (12.6 ov) RV Uthappa, run out
PASS 7-65 (13.4 ov) PP Chawla, stumped
PASS 8-85 (15.6 ov) SP Narine, bowled
PASS 9-103 (17.3 ov) SA Yadav, caught
PASS 10-109 (18.2 ov) UT Yadav, bowled
Wickets credited to bowlers (run outs excluded)

```

3 PP Chawla
3 Sandeep Sharma
3 SP Narine
2 AR Patel
2 MG Johnson
1 JH Kallis
1 L Balaji
1 M Morkel
1 R Dhawan
2 dismissal(s) not credited to any bowler.
All checks passed.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

Applied cleanly, all 22 assertions still pass. Dean's migration is well-written too — it extends `app_user` rather than replacing it, keeps the provider-neutral shape, and the comments state explicitly that authentication never changes approval state. It also answers the scope-model question that was blocking #43: scope is per-competition.

That means **#43 can probably close now** — approved-submitter status and competition scope both exist. Worth checking its criteria against this migration when you get a moment.

Two things to close out:

Close #121. All criteria met.

Your register row for today — the hosting research, ADR-005, the migration, #121. Want it?

Then, if you're still going: batching the ingester is the one remaining piece of your own work that unblocks something real.

yes give me the register

Ran a command

Worth flagging before you paste this: the container reports today as 13 August, but this conversation has moved through the 7th, 9th, 10th and 11th, and I can't tell from here which is right for you. The row below says 2026-08-11 on the assumption that today is still the day of the migration. **Change it if today is actually the 12th or 13th** — a register that dates work wrongly is worse than one that's late.

powershell

@'

"2026-08-11","Ben Swartz","Claude Web","Claude Opus 5","Research; planning; technical
'@ | Add-Content -Path evidence\ai\registers\ben-swartz.csv -Encoding utf8

Check it landed on its own line:

```
powershell
```

```
Get-Content evidence\ai\registers\ben-swartz.csv | Select-Object -Last 2
```

Then export this session as `2026-08-11-continuous-project-session.pdf` into `evidence\ai\transcripts\ben-swartz\` — matching whatever date you settle on — and commit both on a branch:

```
powershell
```

```
git switch -c docs/121-register-ai-usage
git add evidence/ai/registers/ben-swartz.csv evidence/ai/transcripts/ben-swartz/
git commit -m "docs(evidence): register AI usage for the database host migration" -m "
Assisted-by: Claude-Web[Claude Opus 5]"
git push -u origin docs/121-register-ai-usage
```

its the 13th today, its been a long session

Reconciled conflicting timeline details and database migration dates

Then two dates are wrong, and one is already merged.

ADR-005 says `2026-08-11` — I put that in and it's now in `main`. It also says "Verified on 11 August 2026" in the verification section, and the ADR-003 amendment says "Superseded on 11 August 2026". If the migration happened today, all three are wrong by two days.

Worth fixing rather than shrugging at: a decision record with the wrong date undermines the rest of it, and it's the sort of thing a marker checking against commit timestamps would spot immediately.

So, two questions before I give you the corrected register row:

When did the migration actually happen — today, or the 11th? The commit timestamp on ADR-005 will tell you:

```
powershell
```

```
git log -1 --format="%ai %s" -- evidence/decisions/ADR-005-database-host-migration.md
```

And is today's work all of it, or did some happen on the 11th and 12th? If this session has genuinely spanned three days, the register may want two rows rather than one.

Paste the git output and tell me, and I'll give you the register row plus a small `fix(docs)` branch to correct the ADR dates.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git log -1 --format="%ai %s" --  
evidence/decisions/ADR-005-database-host-migration.md  
2026-08-12 17:34:53 +0200 docs(adr): record the database host migration
```

Committed 12 August at 17:34, so ADR-005 is dated a day early throughout — the record says 11 August in three places.

Before correcting it, I need to know when the migration itself happened. You created the project, ran the migrations, seeded and validated — was that the 12th, same day as the commit? Or earlier?

The distinction matters because the ADR has two dates doing different jobs: the decision date in the header, and "Verified on 11 August 2026" in the verification section, which asserts when the migration and validation actually ran.

Tell me which day the migration ran, and whether anything on this list happened on a different day:

- Hosting research and the Oracle/DigitalOcean discussion
- Creating the new project and running migrate/seed/validate
- Writing ADR-005 and creating #121

If it was all the 12th, the fix is one branch replacing "11 August" with "12 August" in ADR-005 and ADR-003, plus a register row dated 2026-08-12 and one dated 2026-08-13 for today. If it spanned days, we split it accordingly.

migration happened on 12th, and db dicussiion was 11th
new project was 12th
ADR 005 and creating #121 was 12th a

Clear. Two register rows, and a date correction to two merged documents.

Step 1 — correct the ADR dates.

powershell

```
git switch main  
git pull origin main  
git switch -c fix/121-correct-adr-dates
```

powershell

```
(Get-Content evidence\decisions\ADR-005-database-host-migration.md -Raw) -replace '202
```

powershell

```
(Get-Content evidence\decisions\ADR-003-database-host-connection-and-migrations.md -Ra
```

Verify nothing was missed, and that no *legitimate* 11 August reference got caught:

powershell

```
Select-String -Path evidence\decisions\ADR-005-database-host-migration.md,evidence\dec
```

Paste that output. There are older dates in ADR-003 — 5 August, 7 August — that must stay untouched, and I want to see they did before you commit.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> (Get-Content
evidence\decisions\ADR-005-database-host-migration.md -Raw) -replace '2026-
08-11','2026-08-12' -replace '11 August 2026','12 August 2026' | Set-Content
evidence\decisions\ADR-005-database-host-migration.md -Encoding utf8
PS C:\Users\BenSwartz\Sport-Analytics-Tool> ^B
: The term " is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and
try again.
At line:1 char:1
+
+ Show more
+ ~
+ CategoryInfo          : ObjectNotFound: (String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\BenSwartz\Sport-Analytics-Tool> (Get-Content
evidence\decisions\ADR-003-database-host-connection-and-migrations.md -
Raw) -replace 'Superseded on 11 August 2026','Superseded on 12 August 2026' |
Set-Content evidence\decisions\ADR-003-database-host-connection-and-
migrations.md -Encoding utf8
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Both replacements ran (the **^B** in between was a stray keystroke, harmless). Now verify:

powershell

```
Select-String -Path evidence\decisions\ADR-005-database-host-migration.md,evidence\dec
```

Paste it. I want to confirm ADR-005 now reads 12 August throughout, and that ADR-003's earlier references — 5 August for the ruling, 7 August for the measurement — are untouched.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
evidence\decisions\ADR-005-database-host-
```

migration.md,evidence\decisions\ADR-003-database-host-connection-and-migrations.md -Pattern "August 2026|2026-08-1"
evidence\decisions\ADR-005-database-host-migration.md:4:- **Date:** 2026-08-12
evidence\decisions\ADR-005-database-host-migration.md:18:The decision had no owner and no date from 5 August 2026. In the intervening
evidence\decisions\ADR-005-database-host-migration.md:126:Verified on 12 August 2026 against the new database: both migrations applied from
evidence\decisions\ADR-003-database-host-connection-and-migrations.md:20:Written clarification was requested from the client on 4 August 2026. The question
evidence\decisions\ADR-003-database-host-connection-and-migrations.md:23:with PostgREST never used. The client confirmed on 5 August 2026 that this is
evidence\decisions\ADR-003-database-host-connection-and-migrations.md:105: A partial measurement was taken on 7 August 2026 over four fixtures and 955
evidence\decisions\ADR-003-database-host-connection-and-migrations.md:121: **Superseded on 12 August 2026 by ADR-005.**
The database moved to a project
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

Exactly right. ADR-005 now reads 12 August in both places; ADR-003's 4, 5 and 7 August references are untouched.

One more in ADR-005 worth checking — line 106 or thereabouts mentioned the partial measurement being taken on 7 August, which is correct and shouldn't have moved. It's not in the output, so either it says something else or the pattern missed it. Fine either way.

Step 2 — add the two register rows.

powershell

@'

```
"2026-08-12","Ben Swartz","Claude Web","Claude Opus 5","Research; planning; technical  
"2026-08-13","Ben Swartz","Claude Web","Claude Opus 5","Review; documentation","Issue  
'@ | Add-Content -Path evidence\ai\registers\ben-swartz.csv -Encoding utf8
```

powershell

```
Get-Content evidence\ai\registers\ben-swartz.csv | Select-Object -Last 3
```

Paste that, then export this session as `2026-08-13-continuous-project-session.pdf` and I'll give you the commit.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
evidence\ai\registers\ben-swartz.csv | Select-Object -Last 3
"2026-08-10","Ben Swartz","Claude Web","Claude Opus 5","Planning; technical
guidance; code generation; documentation; review","Issues #46, #63 and #105:
development seed data, invalid submission examples, entity relationship diagram,
and a database latency measurement","Seed script and committed match fixtures;
three invalid submission examples and the script producing them; registry resolution
fix in ingest-match.ts; docs/database/erd.md; latency measurement script; issue and
Pull Request wording","Verified seed repeatability by running it twice and confirming
zero rows added; confirmed each invalid example is rejected with a message naming
the defect and leaves the database unchanged; identified that the registry guard in
ingest-match.ts was never called, so an unresolvable name was inserted as null and
rejected with an unhelpful constraint error, and corrected it; excluded the source
match files from Prettier rather than reformatting them, because the ingestion hash
records the bytes the source published; measured latency over twenty samples
rather than asserting it, and asked the team to reproduce the figure before acting on
it","evidence/ai/transcripts/ben-swartz/2026-08-10-continuous-project-session.pdf;
Issues #46, #63, #84 and #105 and their linked Pull Requests"
"2026-08-12","Ben Swartz","Claude Web","Claude Opus 5","Research; planning;
technical guidance; documentation","Issue #121: resolve the storage constraint in
ADR-003 consequence 4 and move the project database to an organisation holding a
paid plan","Hosting options comparison; ADR-005; ADR-003 amendment; issue and
Pull Request wording; migration and verification steps","Compared four hosting
options against current published pricing rather than assumption, and rejected an
Oracle proposal by enumerating the PostgreSQL features the schema depends on;
verified the move by applying the migrations from empty, loading the seed, and
confirming reference fixture 729307 passed all twenty-two assertions against the
published scorecard on the new database; corrected a proposed acceptance
criterion after identifying that the previous project cannot be deleted because
authentication still depends on it","evidence/decisions/ADR-005-database-host-
migration.md; evidence/ai/transcripts/ben-swartz/2026-08-13-continuous-project-
session.pdf; Issue #121 and linked Pull Request"
"2026-08-13","Ben Swartz","Claude Web","Claude Opus 5","Review;
documentation","Issue #121: apply the account authorisation migration to the new
database and correct the recorded dates in ADR-003 and ADR-005","Date
correction; verification steps","Re-ran the reference fixture validation after another
member's migration was applied, confirming all twenty-two assertions still passed;
identified that ADR-005 recorded the migration as 11 August when it was performed
on 12 August, and corrected both it and the ADR-003
amendment","evidence/ai/transcripts/ben-swartz/2026-08-13-continuous-project-
session.pdf; Issue #121"
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

```

Both rows landed cleanly on their own lines, properly quoted.

One gap: there's no **11 August** row, but you said the database discussion happened that day — the hosting research, the Oracle and DigitalOcean options, the lecturer's pay-or-move framing. If that used Claude, it needs its own row. If the 11th was just talking to people and the AI work all happened on the 12th, then two rows is right.

Which was it?

Meanwhile, export this session as `2026-08-13-continuous-project-session.pdf` into `evidence\ai\transcripts\ben-swartz\`, then:

powershell

```
git add evidence/decisions evidence/ai/registers/ben-swartz.csv evidence/ai/transcript
```

powershell

```
git commit -m "fix(docs): correct the recorded date of the database host migration" -m
```

Also registers AI usage for 12 and 13 August.

Refs #121

```
Assisted-by: Claude-Web[Claude Opus 5]"
```

powershell

```
git push -u origin fix/121-correct-adr-dates
```

